Implementing Interactive Theorem Prover from Scratch

HIDEAKI TAKAHASHI, Columbia University, USA

TBD

1 PROPOSITIONS AS TYPES

A theorem prover has to decide whether a proof is correct. A proof written in English cannot be checked by a computer, and the Curry–Howard correspondence is exactly the concept that connects logic with programming: *propositions are types, and proofs are programs.* A proposition A is true if we can construct a term t of type A, written $t : A$.

For example, consider the proposition $A \to B$ ("A implies B"). To prove it, we assume A and construct a proof of B. This is exactly a function that takes a value of type A and returns a value of type B. Thus, logical implication corresponds to a function type. Here are three examples, where $\lambda x.\, e$ denotes a function taking x and returning e:

$$\begin{array}{rcll} P \to P & : & \lambda p.\, p & \text{Return the assumption.} \\ A \to B \to A & : & \lambda a.\, \lambda b.\, a & \text{Ignore the second argument.} \\ (A \to B) \to (B \to C) \to A \to C & : & \lambda f.\, \lambda g.\, \lambda a.\, g(f(a)) & \text{Compose two functions.} \end{array}$$

The same idea extends to universal quantification. To prove "for every $x : A$, $B(x)$ holds", we construct a function that takes any $x : A$ and returns a proof of $B(x)$. Unlike an ordinary function type $A \to B$, the return type $B(x)$ may depend on the input x. This is called a *dependent function type*, written

$$\Pi(x : A).\, B(x).$$

For example, "every proposition implies itself" is expressed as

$$\Pi(P{:}\mathrm{Sort}\ 0).\, P \to P,$$

where Sort 0 is the type of propositions. Its proof is simply

$$\lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(p{:}P).\, p.$$

This gives us the central idea behind a theorem prover: *proving a theorem means constructing a term of the required type, and verifying a proof means type-checking that term.* The kernel implements these typing rules rather than reasoning about truth directly.

2 SYNTAX

We begin with a small dependent type theory consisting of five kinds of terms:

$$t, A, B ::= \mathrm{Sort}\ u \mid x \mid \Pi(x{:}A).\, B \mid \lambda(x{:}A).\, t \mid t\ s$$

Here, u is a natural number and x is a variable name. Throughout, $s, t, a, b, f, A, B, C, T$ (possibly with subscripts or primes) stand for terms, x, y, z and the other names bound by Π or λ stand for variable names, and u, v stand for universe levels. Accordingly, the set $\mathrm{FV}(t)$ of free variables of a term t, defined in §9.1, is a set of names.

- Sort u: A universe containing types.
- x: A variable.
- $\Pi(x{:}A).\, B$: A Π-*type* (dependent function type), whose result type B may depend on x.
- $\lambda(x{:}A).\, t$: A λ-*abstraction*, i.e. a function that takes $x : A$ and returns t.

Author's address: Hideaki Takahashi, ht2673@columbia.edu, Columbia University, USA.

- $t\ s$: Function application.

When B does not depend on x, we write $A \to B$ instead of $\Pi(x{:}A).\, B$. Thus, ordinary function types are special cases of dependent function types. For example, the proposition "every proposition implies itself" and its proof are represented as

$$\underbrace{\lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(p{:}P).\, p}_{\text{proof}} \quad : \quad \underbrace{\Pi(P{:}\mathrm{Sort}\ 0).\, P \to P}_{\text{proposition = type of proof}}.$$

Notice that types and expressions share the same grammar. There is no separate syntax for types: whether a term is a valid type is determined by the type checker.

Application associates to the left and arrows associate to the right: $f\ a\ b$ means $(f\ a)\ b$, and $A \to B \to C$ means $A \to (B \to C)$. A function of two arguments is thus a function that takes the first argument and returns a function expecting the second; this encoding is called *currying*. For example, $\lambda(a{:}A).\, \lambda(b{:}B).\, a$ has type $A \to B \to A$, and applied to $x : A$ and $y : B$ it is written $(\lambda(a{:}A).\, \lambda(b{:}B).\, a)\ x\ y$, that is, $((\lambda(a{:}A).\, \lambda(b{:}B).\, a)\ x)\ y$.

3 TYPING RULES

Contexts and judgments. A *context* Γ is an ordered list of assumptions $x_1 : A_1, \ldots, x_n : A_n$. It records the type of each variable in scope, and we write $\mathrm{dom}(\Gamma)$ for the set $\{x_1, \ldots, x_n\}$. Since C-Ext below forbids repeated names, Γ can be used like a dictionary from names to types, and T-Var is simply a lookup. Unlike a plain dictionary, however, the order matters: A_i may mention $x_1, \ldots, x_{i-1}$, as in $P : \mathrm{Sort}\ 0, h : P$. We use two judgments:

- $\vdash \Gamma$ ok: the context Γ is well formed.
- $\Gamma \vdash t : A$: under the assumptions in Γ, the term t has type A.

Read through Curry–Howard, $\Gamma \vdash t : A$ says that t is a proof of A from the hypotheses Γ.

$$\frac{}{\vdash \cdot \ \mathrm{ok}}\ (\textsc{C-Empty}) \qquad \frac{\vdash \Gamma\ \mathrm{ok} \qquad \Gamma \vdash A : \mathrm{Sort}\ u \qquad x \notin \mathrm{dom}(\Gamma)}{\vdash \Gamma, x : A\ \mathrm{ok}}\ (\textsc{C-Ext})$$

$$\frac{\vdash \Gamma\ \mathrm{ok}}{\Gamma \vdash \mathrm{Sort}\ u : \mathrm{Sort}\ (u+1)}\ (\textsc{T-Sort}) \qquad \frac{\vdash \Gamma\ \mathrm{ok} \qquad (x : A) \in \Gamma}{\Gamma \vdash x : A}\ (\textsc{T-Var})$$

$$\frac{\Gamma \vdash A : \mathrm{Sort}\ u \qquad \Gamma, x : A \vdash B : \mathrm{Sort}\ v}{\Gamma \vdash \Pi(x{:}A).\, B : \mathrm{Sort}\ \mathrm{imax}(u, v)}\ (\textsc{T-Pi}) \qquad \frac{\Gamma \vdash A : \mathrm{Sort}\ u \qquad \Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda(x{:}A).\, t : \Pi(x{:}A).\, B}\ (\textsc{T-Lam})$$

$$\frac{\Gamma \vdash f : \Pi(x{:}A).\, B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[x \mapsto a]}\ (\textsc{T-App}) \qquad \frac{\Gamma \vdash t : A \qquad \Gamma \vdash B : \mathrm{Sort}\ u \qquad A \equiv B}{\Gamma \vdash t : B}\ (\textsc{T-Conv})$$

Fig. 1. Well-formed contexts and typing rules.

Two operations used by the rules. Two of the rules rely on operations on terms that we treat as black boxes for now.

- *Substitution.* T-App writes $B[x \mapsto a]$ for B with every free occurrence of x replaced by a. For example, $(P \to P)[P \mapsto Q] = Q \to Q$. Replacing names inside binders needs some care, which §9.1 explains.
- *Definitional equality.* T-Conv writes $A \equiv B$ when A and B are "the same type" in a sense the kernel can decide. Two kinds of differences are ignored. First, the names of bound variables: $\Pi(P{:}\mathrm{Sort}\ 0).\, P \to P$ and $\Pi(Q{:}\mathrm{Sort}\ 0).\, Q \to Q$ state the same proposition. Terms

that differ only in this way are called *α-equivalent*, written $=_\alpha$. Second, computation: if $Id = \lambda(X{:}\text{Sort } 0).\, X$, then $Id\ P$ computes to P by plugging P in for X, so a proof of $Id\ P$ should also count as a proof of P. This computation step, $(\lambda(x{:}A).\, t)\ s \to_\beta t[x \mapsto s]$, is called *β-reduction*.

The kernel decides $A \equiv B$ by computing both sides as far as possible (β-reducing) and then checking whether the results are α-equivalent. §9 defines both operations precisely and gives the algorithms; until then, an intuitive reading is enough.

We read each rule from the bottom up, as a recipe for checking a term.

C-Empty, C-Ext. A context is built one assumption at a time, and each new assumption must have a type as its type. The side condition $x \notin \text{dom}(\Gamma)$ forbids shadowing inside a context. Since the names of bound variables do not matter, this is no restriction on terms: a binder that would shadow can always be renamed first. §9.2 shows why this condition matters.

T-Sort. Every universe lives in the next one; see §10 for why.

T-Var. A variable has the type it was assumed to have. Logically, this is "use a hypothesis".

T-Pi. $\Pi(x{:}A).\, B$ is a type if A is a type and B is a type once we assume $x : A$. It lives in the larger of the two universes, except that it is a proposition whenever B is one; $\text{imax}(u, v)$ is 0 if $v = 0$ and $\max(u, v)$ otherwise (§10.3). Logically, this rule says when $A \to B$ and $\forall x{:}A.\, B$ are well-formed propositions.

T-Lam. To build a function from A to B, assume $x : A$ and build $t : B$. Logically, this is the introduction rule for $\to$ and $\forall$: to prove $A \to B$, assume A and prove B. Note that B may mention x, and it is kept as is in the resulting Π type.

T-App. If f maps each $x : A$ to an element of B, and $a : A$, then $f\ a$ has type B with x replaced by a. Logically, this is modus ponens when B does not mention x, and instantiation of a universal statement when it does. For example, if $f : \Pi(n{:}\text{Nat}).\, n + 0 = n$, then $f\ 3 : 3 + 0 = 3$ (natural numbers are added in §7).

T-Conv. Types that are equal up to computation are interchangeable. For example, from $h : Id\ P$ we may conclude $h : P$. This is the only rule that is not determined by the shape of t.

Why application matters. Without T-App, a hypothesis can be returned but never used. Roughly, the provable propositions are then those whose conclusion is literally one of the hypotheses, such as $P \to Q \to P$. Composition, $(P \to Q) \to (Q \to R) \to P \to R$, has the proof $\lambda(f{:}P \to Q).\, \lambda(g{:}Q \to R).\, \lambda(p{:}P).\, g\ (f\ p)$, which needs application.

4 THE TYPING RULES AS A LOGIC

The typing rules of §3 are meant to be rules of logic. This section makes that claim precise, and identifies the property, consistency, that a trustworthy kernel must have.

Consider the logic whose formulas are built from proposition variables $P, Q, \ldots$ by implication and quantification over propositions:

$$\varphi, \psi ::= P \mid \varphi \to \psi \mid \forall P.\, \varphi$$

This is second-order propositional logic. Its proofs are natural-deduction derivations of judgments $\Delta \vdash \varphi$, where Δ is a list of hypotheses. Each formula translates to a type, and each list of hypotheses to a context:

$$P^* = P, \qquad (\varphi \to \psi)^* = \varphi^* \to \psi^*, \qquad (\forall P.\, \varphi)^* = \Pi(P{:}\text{Sort } 0).\, \varphi^*,$$

$$(\varphi_1, \ldots, \varphi_n)^* = P_1 : \text{Sort } 0, \ldots, P_k : \text{Sort } 0,\ h_1 : \varphi_1^*, \ldots, h_n : \varphi_n^*,$$

where $P_1, \ldots, P_k$ are the proposition variables free in Δ or in the conclusion φ of the judgment $\Delta \vdash \varphi$ being translated, and $h_1, \ldots, h_n$ are fresh names for the hypotheses. The conclusion must be

included because a proposition variable may occur only there, as P does in $\cdot \vdash P \to P$. Fig. 2 places each logical rule next to the typing rule it corresponds to; in each row, Δ^* is the context for the conclusion of the rule. Erasing the terms from the right column gives the left column exactly.

	Logic	**Type theory**
Hypothesis	$\dfrac{\varphi_i \in \Delta}{\Delta \vdash \varphi_i}$	$\dfrac{(h_i : \varphi_i^*) \in \Delta^*}{\Delta^* \vdash h_i : \varphi_i^*}$ (T-Var)
$\to$-intro	$\dfrac{\Delta, \varphi \vdash \psi}{\Delta \vdash \varphi \to \psi}$	$\dfrac{\Delta^*, h : \varphi^* \vdash t : \psi^*}{\Delta^* \vdash \lambda(h{:}\varphi^*).\, t : \varphi^* \to \psi^*}$ (T-Lam)
$\to$-elim	$\dfrac{\Delta \vdash \varphi \to \psi \qquad \Delta \vdash \varphi}{\Delta \vdash \psi}$	$\dfrac{\Delta^* \vdash f : \varphi^* \to \psi^* \qquad \Delta^* \vdash a : \varphi^*}{\Delta^* \vdash f\ a : \psi^*}$ (T-App)
$\forall$-intro	$\dfrac{\Delta \vdash \varphi \qquad P \notin \mathrm{FV}(\Delta)}{\Delta \vdash \forall P.\, \varphi}$	$\dfrac{\Delta^*, P : \mathrm{Sort}\ 0 \vdash t : \varphi^*}{\Delta^* \vdash \lambda(P{:}\mathrm{Sort}\ 0).\, t : \Pi(P{:}\mathrm{Sort}\ 0).\, \varphi^*}$ (T-Lam)
$\forall$-elim	$\dfrac{\Delta \vdash \forall P.\, \varphi}{\Delta \vdash \varphi[P \mapsto \psi]}$	$\dfrac{\Delta^* \vdash f : \Pi(P{:}\mathrm{Sort}\ 0).\, \varphi^* \qquad \Delta^* \vdash \psi^* : \mathrm{Sort}\ 0}{\Delta^* \vdash f\ \psi^* : \varphi^*[P \mapsto \psi^*]}$ (T-App)

Fig. 2. Natural deduction (left) and the typing rules that implement it (right). Premises that only check that a type is well formed are omitted.

In the $\forall$-intro row, the side condition $P \notin \mathrm{FV}(\Delta)$ guarantees that no hypothesis mentions P, so the typing rule may add $P : \mathrm{Sort}\ 0$ after the hypotheses. In the $\forall$-elim row, the premise $\psi^* : \mathrm{Sort}\ 0$ holds for every formula ψ, even one that contains $\forall$, because T-Pi places every translated formula in Sort 0 (§10.3). Hence $\forall$-elim may instantiate P with any formula, as in the logic.

Theorem 1 (Proofs are terms). *If $\Delta \vdash \varphi$ is provable, then $\Delta^* \vdash t : \varphi^*$ for some term t.*

The figure turns each step of a proof into a step of a typing derivation, so every proof becomes a term. Appendix A states the theorem precisely and gives the full proof.

The theorem says that the type checker can check every proof of the logic. For the type checker to be trustworthy, we also need the opposite guarantee: it must not accept a term for a proposition that the logic refutes. Define $\bot = \forall P.\, P$ ("everything holds"). No proof of $\bot$ exists: if each P is read as true or false, every rule preserves truth under every reading, but $\forall P.\, P$ is false. The type theory should agree, that is, no closed term should have type

$$\bot^* = \Pi(P{:}\mathrm{Sort}\ 0).\, P.$$

A closed term $t : \bot^*$ would prove every proposition, since $t\ C : C$ for every $C : \mathrm{Sort}\ 0$. A system in which no closed term has type $\bot^*$ is called *consistent*, and a design for universes is acceptable only if the system remains consistent (§10). §11 states consistency for our system as Theorem 7.

One might hope for the full converse of Theorem 1: every term of type φ^* comes from a proof of φ. That is a stronger property, called conservativity, and it is harder to establish here, because Sort 0 also contains types that are not translations of formulas, such as $\Pi(F{:}\mathrm{Sort}\ 0 \to \mathrm{Sort}\ 0).\, F\ P \to F\ P$, and a term may pass through them. We do not need it; consistency is enough.

5 A TYPE CHECKING ALGORITHM

The rules in Fig. 1 define which judgments hold, but they are not yet an algorithm. The first five typing rules are triggered by the shape of the term: a Sort , a variable, a Π, a λ, or an application. T-Conv has no such trigger; it may be applied at any point, so a program cannot tell when to use it. The standard solution splits type checking into two mutually recursive functions [5]: Infer computes the type of a term, and Check verifies a term against an expected type. Definitional equality is used only where two types must be compared: in Check, and when a type must have a particular shape (a Π or a Sort).

The algorithm calls two functions on terms, whose definitions we postpone to §9:

- Whnf(T) computes T until its outermost shape is visible. For example, Whnf($Id\ (P \to P)) = P \to P$, which is a Π.
- IsDefEq(A, B) decides $A \equiv B$.

Algorithm 1 Type inference and checking. Whnf and IsDefEq are defined in Algorithm 2.

```
 1: function Infer(Γ, t)
 2:     if t = Sort u then return Sort (u + 1)                    ▷ T-Sort
 3:     else if t = x then                                        ▷ T-Var
 4:         if (x : A) ∈ Γ then return A
 5:         else fail "unbound variable"
 6:     else if t = Π(x:A). B then                                ▷ T-Pi
 7:         u ← InferSort(Γ, A)
 8:         (x′, B′) ← Freshen(Γ, x, B)
 9:         v ← InferSort((Γ, x′ : A), B′)
10:         return Sort imax(u, v)
11:     else if t = λ(x:A). s then                                ▷ T-Lam
12:         InferSort(Γ, A)
13:         (x′, s′) ← Freshen(Γ, x, s)
14:         B ← Infer((Γ, x′ : A), s′)
15:         return Π(x′:A). B
16:     else if t = f a then                                      ▷ T-App
17:         T ← Whnf(Infer(Γ, f))
18:         if T = Π(x:A). B then
19:             Check(Γ, a, A)
20:             return B[x ↦ a]
21:         else fail "not a function"

22: function Check(Γ, t, T)                                       ▷ T-Conv
23:     T′ ← Infer(Γ, t)
24:     if IsDefEq(T′, T) then return
25:     else fail "type mismatch"

26: function InferSort(Γ, A)
27:     T ← Whnf(Infer(Γ, A))
28:     if T = Sort u then return u
29:     else fail "expected a type"

30: function Freshen(Γ, x, b)
31:     if x ∈ dom(Γ) then
32:         choose x′ ∉ dom(Γ) ∪ FV(b)
33:         return (x′, b[x ↦ x′])
34:     else return (x, b)
```

Algorithm 1 has the following properties.

- Each case of Infer corresponds to exactly one rule of Fig. 1, so the algorithm is *syntax-directed*. Finding a proof may be hard, but checking a given proof is mechanical.
- T-Conv is never applied on its own. It happens inside Check, the only place where an inferred type meets an expected one.
- InferSort implements the premises of the form $\Gamma \vdash A : \mathrm{Sort}\ u$. It is also how the kernel decides whether a term is a type at all: A is a type exactly when its type computes to a universe. A λ-abstraction can never be a type, because its inferred type is always a Π.
- Freshen renames a binder whose name is already in the context, implementing the side condition of C-Ext. Here $\mathrm{FV}(b)$ is the set of free variables of b (§9.1); §9.2 shows what goes wrong without this step.
- Starting from the empty context, every context the algorithm builds is well formed, because each assumption is added only after InferSort succeeds. A caller that starts from a non-empty context must check that context separately.
- Termination depends on Whnf and IsDefEq terminating, which holds for well-typed terms by strong normalization (§11).

6 WORKED EXAMPLES

This section runs Algorithm 1 on a few small terms and relates each run to the typing rules of Fig. 1. To verify a proof t of a theorem T, the kernel calls $\textsc{Check}(\cdot, t, T)$.

The identity proof. Let $\Gamma_1 = P : \mathrm{Sort}\ 0$ and $\Gamma_2 = \Gamma_1, h : P$. Omitting the well-formedness premises $\vdash \Gamma$ ok, the derivation of the opening example is

$$\dfrac{\dfrac{}{\cdot \vdash \mathrm{Sort}\ 0 : \mathrm{Sort}\ 1}\ \text{T-Sort} \qquad \dfrac{\dfrac{}{\Gamma_1 \vdash P : \mathrm{Sort}\ 0}\ \text{T-Var} \qquad \dfrac{}{\Gamma_2 \vdash h : P}\ \text{T-Var}}{\Gamma_1 \vdash \lambda(h{:}P).\, h : P \to P}\ \text{T-Lam}}{\cdot \vdash \lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, h : \Pi(P{:}\mathrm{Sort}\ 0).\, P \to P}\ \text{T-Lam}$$

The algorithm builds the same derivation from the bottom up. $\textsc{Infer}(\cdot, \lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, h)$ makes the following calls:

1. $\textsc{InferSort}(\cdot, \mathrm{Sort}\ 0)$: Infer returns Sort 1, so the annotation is a type. (T-Sort)
2. Freshen leaves P unchanged, since the context is empty.
3. $\textsc{Infer}(\Gamma_1, \lambda(h{:}P).\, h)$, which makes three calls of its own:
   (a) $\textsc{InferSort}(\Gamma_1, P)$: Infer looks up P and returns Sort 0. (T-Var)
   (b) Freshen leaves h unchanged.
   (c) $\textsc{Infer}(\Gamma_2, h)$ looks up h and returns P. (T-Var)

   It returns $\Pi(h{:}P).\, P$. (inner T-Lam)

The outer call returns $\Pi(P{:}\mathrm{Sort}\ 0).\, \Pi(h{:}P).\, P$. (outer T-Lam)

Each call that returns a type corresponds to one node of the tree, and no call uses computation. Finally, Check compares the result with the stated type. The two differ only in the name of one bound variable, because $P \to P$ abbreviates $\Pi(_{:}P).\, P$ and neither h nor _ occurs in the body, so IsDefEq succeeds and the proof is accepted.

A wrong proof. Suppose a user claims that $t = \lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, P$ proves $\Pi(P{:}\mathrm{Sort}\ 0).\, P \to P$. The kernel calls $\textsc{Check}(\cdot, t, \Pi(P{:}\mathrm{Sort}\ 0).\, P \to P)$, which first calls Infer. The calls are the same as above, except that the last one, $\textsc{Infer}(\Gamma_2, P)$, returns Sort 0. So Infer succeeds with

$$\textsc{Infer}(\cdot, t) = \Pi(P{:}\mathrm{Sort}\ 0).\, \Pi(h{:}P).\, \mathrm{Sort}\ 0.$$

The term t is well typed; it is just not a proof of the claimed proposition. Check then compares the two types. Neither can compute further, and no renaming of bound variables makes them equal: the innermost body is Sort 0 on one side and P on the other. So IsDefEq fails and Check reports a

type mismatch. (§9.3 replays this comparison step by step with the algorithm that IsDefEq uses.) The error is found only because the user stated the proposition to be proved. A kernel that only ran Infer would accept t and silently assign it a different type.

Using a hypothesis: modus ponens. The examples so far never apply a function. Consider

$$\begin{aligned} & t = \lambda(P{:}\text{Sort } 0).\, \lambda(Q{:}\text{Sort } 0).\, \lambda(f{:}P \to Q).\, \lambda(p{:}P).\, f\ p \\ \text{against} \quad & \Pi(P{:}\text{Sort } 0).\, \Pi(Q{:}\text{Sort } 0).\, (P \to Q) \to P \to Q, \end{aligned}$$

which says "if P implies Q and P holds, then Q holds". The four λs extend the context to $\Gamma = P : \text{Sort } 0, Q : \text{Sort } 0, f : P \to Q, p : P$ exactly as before. The body $f\ p$ reaches the application case of Infer:

1. $\textsc{Infer}(\Gamma, f)$ returns $P \to Q$, that is, $\Pi(_{:}P).\, Q$. Whnf leaves it unchanged, and it is a Π.
2. $\textsc{Check}(\Gamma, p, P)$ succeeds, since $p : P$ is in Γ.
3. The result is $Q[_ \mapsto p] = Q$. The name _ does not occur in Q, so the substitution does nothing. (T-App)

If the user had written $f\ f$ instead, step 2 would call $\textsc{Check}(\Gamma, f, P)$, which fails because $P \to Q$ is not P.

Instantiating a universal statement. In a dependent application, the substitution in T-App does real work. Let $id = \lambda(P{:}\text{Sort } 0).\, \lambda(h{:}P).\, h$ be the proof from the first example, and let $\Gamma = Q : \text{Sort } 0$. Then $\textsc{Infer}(\Gamma, id\ Q)$:

1. infers $\Pi(P{:}\text{Sort } 0).\, \Pi(h{:}P).\, P$ for id, which is already a Π;
2. checks the argument, $\textsc{Check}(\Gamma, Q, \text{Sort } 0)$;
3. returns $(\Pi(h{:}P).\, P)[P \mapsto Q] = \Pi(h{:}Q).\, Q$, that is, $Q \to Q$. (T-App)

Logically, this is $\forall$-elimination: from "every proposition implies itself" we obtain "Q implies Q". The type of the result depends on the *value* of the argument Q; this is exactly what the dependent Π type expresses.

Conversion. Recall $Id = \lambda(X{:}\text{Sort } 0).\, X$ from §3. Our system has no definitions, so $Id\ P$ below is shorthand for the term $(\lambda(X{:}\text{Sort } 0).\, X)\ P$. Consider checking

$$t = \lambda(P{:}\text{Sort } 0).\, \lambda(h{:}Id\ P).\, h \quad \text{against} \quad \Pi(P{:}\text{Sort } 0).\, P \to P.$$

Infer first checks that the annotation $Id\ P$ is a type. $\textsc{InferSort}(\Gamma_1, Id\ P)$ reaches the application case of Infer: the function $\lambda(X{:}\text{Sort } 0).\, X$ has type $\Pi(X{:}\text{Sort } 0).\, \text{Sort } 0$, the argument P passes $\textsc{Check}(\Gamma_1, P, \text{Sort } 0)$, and the result is $\text{Sort } 0[X \mapsto P] = \text{Sort } 0$. The body h then has type $Id\ P$, so

$$\textsc{Infer}(\cdot, t) = \Pi(P{:}\text{Sort } 0).\, \Pi(h{:}Id\ P).\, Id\ P.$$

Renaming bound variables alone does not turn this into $\Pi(P{:}\text{Sort } 0).\, P \to P$. But IsDefEq computes both sides before comparing them, and $Id\ P \to_\beta P$, so the inferred type becomes $\Pi(P{:}\text{Sort } 0).\, \Pi(h{:}P).\, P$ and the comparison succeeds as in the first example. In the typing rules, this step is an application of T-Conv at the root of the derivation:

$$\frac{\cdot \vdash t : \Pi(P{:}\text{Sort } 0).\, Id\ P \to Id\ P \qquad \cdot \vdash \Pi(P{:}\text{Sort } 0).\, P \to P : \text{Sort } 0 \qquad \Pi(P{:}\text{Sort } 0).\, Id\ P \to Id\ P \equiv \Pi(P{:}\text{Sort } 0).\, P \to P}{\cdot \vdash t : \Pi(P{:}\text{Sort } 0).\, P \to P}\ \text{T-Conv}$$

The algorithm never decides where to apply T-Conv. As described in §5, conversion is hidden in the call to IsDefEq, which runs only where two types are compared.

Exercises.

1. Prove $\Pi(P{:}\mathrm{Sort}\ 0).\,\Pi(Q{:}\mathrm{Sort}\ 0).\,P \to Q \to P$. One answer is
$$\lambda(P{:}\mathrm{Sort}\ 0).\,\lambda(Q{:}\mathrm{Sort}\ 0).\,\lambda(p{:}P).\,\lambda(q{:}Q).\,p.$$
The proof never uses q; this system allows hypotheses to go unused.
2. Prove that implication is transitive:
$$\Pi(P{:}\mathrm{Sort}\ 0).\,\Pi(Q{:}\mathrm{Sort}\ 0).\,\Pi(R{:}\mathrm{Sort}\ 0).\,(P \to Q) \to (Q \to R) \to P \to R.$$
One answer ends in $\lambda(f{:}P \to Q).\,\lambda(g{:}Q \to R).\,\lambda(p{:}P).\,g\ (f\ p)$; it uses the application case twice.
3. Explain why $\lambda(P{:}\mathrm{Sort}\ 0).\,\lambda(h{:}P).\,h\ h$ is rejected, and at which line of Algorithm 1.

Running Infer on these by hand, as in the examples above, is a good check of one's understanding of the algorithm.

7 INDUCTIVE TYPES: FROM NATURAL NUMBERS TO EQUALITY

So far the only types are universes and Π types. This is enough for propositional logic, but not for mathematics: there are no numbers to reason about, and no way to state that two things are equal. Lean provides both through a single mechanism, *inductive types*. We start with the natural numbers and ask two questions: how do we compute with them, and how do we prove things about them? The answers turn out to be the same constant, the *recursor* (§7.2). We then generalize step by step, to more constructors (§7.3), and to parameters and indices (§7.4), and arrive at a procedure that works for any inductive declaration (§7.5). Finally, we will see that even equality is an inductive type (§7.6).

7.1 Constants and Definitions

First we need a way to add named objects to the kernel. Terms gain *constants* c:

$$t, A, B ::= \cdots \mid c$$

A *signature* Σ lists the constants. Each entry either declares a constant with its type, written $c : C$, or also gives it a body, written $c : C := t$; the latter are *definitions*. For example, Nat : Sort 1 below is a declaration, and add in §7.2 is a definition. Before an entry is added, its type is checked to be a type in the empty context, and its body, if any, is checked against that type, as Lean does for each `def`.

The signature is used in two ways. First, a constant has the type the signature gives it:

$$\frac{\vdash \Gamma\ \mathsf{ok} \qquad (c : C) \in \Sigma}{\Gamma \vdash c : C}\ (\text{T-Const})$$

where $(c : C) \in \Sigma$ covers both declarations and definitions. Second, a definition can be *unfolded*: the constant is replaced by its body. This computation step is called δ*-reduction*:

$$c \to_\delta t \qquad \text{if } (c : C := t) \in \Sigma.$$

For example, add 2 1 $\to_\delta$ $(\lambda(m{:}\mathrm{Nat}).\,\lambda(n{:}\mathrm{Nat}).\,\cdots)$ 2 1, after which β-reduction continues. A declared constant without a body, such as Nat or zero, never δ-reduces. Like β, the step may be applied to any subterm.

Constants are not variables: they are never bound, so $\mathrm{FV}(c) = \emptyset$ and substitution leaves them unchanged. Reduction now consists of β, δ, and the ι-rules introduced below, and conversion (§9.4) is taken with respect to all three.

7.2 Natural Numbers: Recursion and Induction

In Lean, the natural numbers are declared as

```
inductive Nat where
  | zero : Nat
  | succ (n : Nat) : Nat
```

The declaration says that every natural number is built in one of two ways: it is zero, or it is succ n, the successor of a natural number n. For example, 2 is succ (succ zero). In our kernel, the declaration adds three constants without bodies:

$$\mathsf{Nat} : \mathsf{Sort}\ 1, \qquad \mathsf{zero} : \mathsf{Nat}, \qquad \mathsf{succ} : \mathsf{Nat} \to \mathsf{Nat}.$$

zero and succ are called *constructors*. They build values but never compute: succ (succ zero) is already in normal form, and it *is* the number 2. We write 0, 1, 2, . . . for zero, succ zero, succ (succ zero), and so on.

This gives us numbers, but nothing to do with them. A function such as addition must look at a number and act differently on zero and on succ n, and our terms have no way to do that yet.

Computing: addition. How should addition be defined? In Lean one writes it by pattern matching and recursion on the second argument:

```
def add : Nat -> Nat -> Nat
  | m, zero   => m
  | m, succ k => succ (add m k)
```

that is,

$$\mathsf{add}\ m\ \mathsf{zero} = m, \qquad \mathsf{add}\ m\ (\mathsf{succ}\ k) = \mathsf{succ}\ (\mathsf{add}\ m\ k).$$

Our kernel has neither pattern matching nor recursive definitions, and it should not get general recursion: a definition $\mathit{bad} : \Pi(P{:}\mathsf{Sort}\ 0).\, P := \mathit{bad}$ would type-check and prove False.

Instead, look at the shape of the definition. For a fixed m, the function add m is determined by two pieces of data: its value on zero, which is m, and a way to turn its value r on k into its value on succ k, which is succ r. Every function $f : \mathsf{Nat} \to C$ defined by recursion of this kind, called *structural recursion*, has the same shape:

$$f\ \mathsf{zero} = z, \qquad f\ (\mathsf{succ}\ k) = s\ k\ (f\ k),$$

for some $z : C$ and $s : \mathsf{Nat} \to C \to C$; the step s receives both k and the result $f\ k$ of the recursive call. So instead of allowing recursive definitions, we add one constant that builds f from z and s, once and for all:

$$\mathsf{rec}_C : C \to (\mathsf{Nat} \to C \to C) \to \mathsf{Nat} \to C,$$

together with two computation rules, called ι*-rules*, which are exactly the equations above:

$$\mathsf{rec}_C\ z\ s\ \mathsf{zero} \ \to_\iota\ z, \qquad \mathsf{rec}_C\ z\ s\ (\mathsf{succ}\ n) \ \to_\iota\ s\ n\ (\mathsf{rec}_C\ z\ s\ n).$$

Addition is now an ordinary, non-recursive definition:

$$\mathsf{add} : \mathsf{Nat} \to \mathsf{Nat} \to \mathsf{Nat} := \lambda(m{:}\mathsf{Nat}).\, \lambda(n{:}\mathsf{Nat}).\, \mathsf{rec}_{\mathsf{Nat}}\ m\ s\ n, \qquad s = \lambda(k{:}\mathsf{Nat}).\, \lambda(r{:}\mathsf{Nat}).\, \mathsf{succ}\ r.$$

We write $m + n$ for add m n. The kernel computes $2 + 1$ with δ, β, and ι steps only:

$$\begin{aligned} 2 + 1 &\to_{\delta\beta} \mathsf{rec}_{\mathsf{Nat}}\ 2\ s\ (\mathsf{succ}\ 0) \ \to_\iota\ s\ 0\ (\mathsf{rec}_{\mathsf{Nat}}\ 2\ s\ 0) \\ &\to_\beta \mathsf{succ}\ (\mathsf{rec}_{\mathsf{Nat}}\ 2\ s\ 0) \ \to_\iota\ \mathsf{succ}\ 2 = 3. \end{aligned}$$

Other functions follow the same pattern. Doubling, with $\mathit{double}\ (\mathsf{succ}\ k) = \mathsf{succ}\ (\mathsf{succ}\ (\mathit{double}\ k))$, is

$$\mathit{double} := \lambda(n{:}\mathsf{Nat}).\, \mathsf{rec}_{\mathsf{Nat}}\ \mathsf{zero}\ (\lambda(k{:}\mathsf{Nat}).\, \lambda(r{:}\mathsf{Nat}).\, \mathsf{succ}\ (\mathsf{succ}\ r))\ n,$$

and *double* 2 ⟶ succ (succ (*double* 1)) ⟶ succ^4 (*double* 0) ⟶ succ^4 zero = 4.

The interesting point is that no new recursion mechanism is needed. The constants of the inductive type, together with rec_C and its two rules, are enough to express every structurally recursive function on Nat. And unlike general recursion, this one always terminates: each ι-step removes one succ from the argument.

Proving: induction. Now consider proofs. To prove that a property $P\ n$ holds for every n, where P : Nat ⟶ Sort 0, mathematics uses induction: prove the *base case* P zero, and the *step* $\Pi(k{:}\mathrm{Nat}).\ P\ k \to P\ (\mathrm{succ}\ k)$, whose argument $P\ k$ is the *induction hypothesis*. Through Curry–Howard, a proof of $\Pi(n{:}\mathrm{Nat}).\ P\ n$ is a function f that takes n and returns a proof of $P\ n$. How do we build such an f from *base* : P zero and *step*? For 3, the proof is *step* 2 (*step* 1 (*step* 0 *base*)), and in general

$$f\ \mathrm{zero} = \mathit{base}, \qquad f\ (\mathrm{succ}\ k) = \mathit{step}\ k\ (f\ k).$$

This is a structurally recursive function again, with exactly the shape of add above: z = *base* and s = *step*. Induction is recursion that returns proofs.

There is one difference. For add, the result type C was the same for every n. Here it depends on n: f zero : P zero, f 1 : P 1, and so on. So rec_C, whose type mentions a fixed C, cannot express induction as it stands.

One recursor for both. Putting the two cases side by side shows what to generalize:

	Recursion	**Induction**
result for n	C	$P\ n$
case zero	$z : C$	*base* : P zero
case succ	$s : \Pi(k{:}\mathrm{Nat}).\ C \to C$	*step* : $\Pi(k{:}\mathrm{Nat}).\ P\ k \to P\ (\mathrm{succ}\ k)$
result	Nat ⟶ C	$\Pi(n{:}\mathrm{Nat}).\ P\ n$

The left column is the special case of the right column in which $P\ n$ is the same type C for every n. So we take the right column, with P replaced by an arbitrary function M : Nat ⟶ Sort u that gives the type of the result for each n. This function is called the *motive*. The result is the *recursor* $\mathrm{Nat.rec}_u$ of Fig. 3. Its arguments are the motive, one *minor premise* for each constructor, and finally the number n to recurse on, called the *major premise*. Its ι-rules are those of rec_C. As in Lean, where the recursor is `Nat.rec.{u}`, there is one recursor for each universe level; since we have no universe polymorphism, the level is a subscript.

Constant and type	**Lean**
Nat : Sort 1	`Nat`
zero : Nat	`Nat.zero`
succ : Nat ⟶ Nat	`Nat.succ`
$\mathrm{Nat.rec}_u$: $\Pi(M{:}\mathrm{Nat} \to \mathrm{Sort}\ u).\ M\ \mathrm{zero} \to (\Pi(n{:}\mathrm{Nat}).\ M\ n \to M\ (\mathrm{succ}\ n)) \to \Pi(n{:}\mathrm{Nat}).\ M\ n$	`Nat.rec`

$$\mathrm{Nat.rec}_u\ M\ z\ s\ \mathrm{zero} \to_\iota z$$
$$\mathrm{Nat.rec}_u\ M\ z\ s\ (\mathrm{succ}\ n) \to_\iota s\ n\ (\mathrm{Nat.rec}_u\ M\ z\ s\ n)$$

Fig. 3. The constants for natural numbers (top) and the ι-rules of the recursor (bottom).

The one recursor does both jobs, depending on the motive.

- *Recursion.* With the constant motive $\lambda(_{:}\mathsf{Nat}).\, C$ at level 1, the type of $\mathsf{Nat.rec}_1\ (\lambda(_{:}\mathsf{Nat}).\, C)$ is convertible to the type of rec_C. So rec_C is just $\mathsf{Nat.rec}_1\ (\lambda(_{:}\mathsf{Nat}).\, C)$, and from now on

  $$\mathsf{add} := \lambda(m{:}\mathsf{Nat}).\, \lambda(n{:}\mathsf{Nat}).\, \mathsf{Nat.rec}_1\ (\lambda(_{:}\mathsf{Nat}).\, \mathsf{Nat})\ m\ (\lambda(k{:}\mathsf{Nat}).\, \lambda(r{:}\mathsf{Nat}).\, \mathsf{succ}\ r)\ n,$$

  which is the definition in `examples/zero-add.leanmini`.
- *Induction.* With a predicate $M : \mathsf{Nat} \to \mathsf{Sort}\ 0$ at level 0, the type of $\mathsf{Nat.rec}_0$ is the induction principle

  $$M\ \mathsf{zero} \to (\forall n.\ M\ n \to M\ (\mathsf{succ}\ n)) \to \forall n.\ M\ n.$$

  Here the choice of imax (§10.3) is essential. The conclusion $\Pi(n{:}\mathsf{Nat}).\, M\ \ n$ has type $\mathsf{Sort}\ \mathrm{imax}(1,0) = \mathsf{Sort}\ 0$, so "$M\ \ n$ for every n" is a proposition although Nat lives in $\mathsf{Sort}\ 1$. With max, it would live in $\mathsf{Sort}\ 1$, and statements about numbers would not be propositions.

The type of $\mathsf{Nat.rec}_u$ is not arbitrary: it can be read off the declaration of Nat. There is one minor premise per constructor. Each minor premise takes the constructor's arguments, plus one induction hypothesis for each argument whose type is Nat itself (a *recursive* argument), and returns the motive applied to the constructor. zero has no arguments, so its case is just $M\ \mathsf{zero}$; succ has one recursive argument n, so its case is $\Pi(n{:}\mathsf{Nat}).\, M\ n \to M\ (\mathsf{succ}\ n)$. The rest of this section applies this recipe to more and more general declarations.

7.3 More Constructors: Trees

Nat has two constructors, and only one of them has a recursive argument. Consider a type with three constructors:

```
inductive Tree where
  | leaf   : Tree
  | node   : Tree -> Tree
  | branch : Tree -> Tree -> Tree
```

For example, branch (node leaf) leaf is a tree with four constructors. To build $M\ t$ for every tree t, where $M : \mathsf{Tree} \to \mathsf{Sort}\ v$, the recipe asks for one case per constructor, and each argument of type Tree contributes one hypothesis:

$$\begin{aligned} m_{\mathsf{leaf}} &: M\ \mathsf{leaf} \\ m_{\mathsf{node}} &: \Pi(t{:}\mathsf{Tree}).\, M\ t \to M\ (\mathsf{node}\ t) \\ m_{\mathsf{branch}} &: \Pi(l{:}\mathsf{Tree}).\, \Pi(r{:}\mathsf{Tree}).\, M\ l \to M\ r \to M\ (\mathsf{branch}\ l\ r) \end{aligned}$$

The recursor takes the motive, the three cases, and a tree:

$$\mathsf{Tree.rec}_v : \Pi(M{:}\mathsf{Tree} \to \mathsf{Sort}\ v).\ \ M\ \mathsf{leaf} \to (\cdots_{\mathsf{node}}) \to (\cdots_{\mathsf{branch}}) \to \Pi(t{:}\mathsf{Tree}).\, M\ t,$$

where the dots stand for the types of m_{node} and m_{branch}. Compared with $\mathsf{Nat.rec}_v$, only the number of cases and the number of hypotheses per case have changed. Writing R for the partial application $\mathsf{Tree.rec}_v\ M\ m_{\mathsf{leaf}}\ m_{\mathsf{node}}\ m_{\mathsf{branch}}$, the ι-rules are

$$\begin{aligned} R\ \mathsf{leaf} &\to_\iota m_{\mathsf{leaf}} \\ R\ (\mathsf{node}\ t) &\to_\iota m_{\mathsf{node}}\ t\ (R\ t) \\ R\ (\mathsf{branch}\ l\ r) &\to_\iota m_{\mathsf{branch}}\ l\ r\ (R\ l)\ (R\ r). \end{aligned}$$

Each rule selects the case for the constructor at the head of the major premise, and fills each hypothesis with a recursive call on the corresponding subtree. For example, with $M = \lambda(_{:}\mathsf{Tree}).\, \mathsf{Nat}$ and the cases

$$\begin{aligned} m_{\mathsf{leaf}} &= 1, \qquad m_{\mathsf{node}} = \lambda(t{:}\mathsf{Tree}).\, \lambda(i{:}\mathsf{Nat}).\, 1 + i, \\ m_{\mathsf{branch}} &= \lambda(l{:}\mathsf{Tree}).\, \lambda(r{:}\mathsf{Tree}).\, \lambda(i{:}\mathsf{Nat}).\, \lambda(j{:}\mathsf{Nat}).\, 1 + i + j, \end{aligned}$$

R counts the constructors of a tree, and

$$R\ (\mathsf{branch}\ (\mathsf{node}\ \mathsf{leaf})\ \mathsf{leaf}) \to 1{+}R\ (\mathsf{node}\ \mathsf{leaf}){+}R\ \mathsf{leaf} \to 1{+}(1{+}R\ \mathsf{leaf}){+}1 \to 1{+}(1{+}1){+}1 = 4.$$

With M a predicate ($v = 0$), the same recursor is structural induction on trees. In general, a type with n constructors gets a recursor with n minor premises $m_1, \ldots, m_n$, and the kernel needs no code specific to the type: it only needs to know, for each constructor, which case to pick and which arguments to recurse on.

7.4 Parameters and Indices: Lists and Vectors

Parameters: lists. Next, consider lists. A list of numbers is either empty or a number in front of a list of numbers, so we could declare a type ListNat with constructors nil and cons : Nat → ListNat → ListNat. But then lists of trees would need another declaration, ListTree, with the same constructors and a separate recursor, and so on for every element type. It is better to make the element type an argument of the type itself:

```
inductive List (A : Sort 1) where
  | nil  : List A
  | cons : A -> List A -> List A
```

Now List Nat and List Tree are two instances of one declaration. The declaration adds

$$\mathsf{List} : \mathsf{Sort}\ 1 \to \mathsf{Sort}\ 1, \qquad \mathsf{nil} : \Pi(A{:}\mathsf{Sort}\ 1).\ \mathsf{List}\ A,$$
$$\mathsf{cons} : \Pi(A{:}\mathsf{Sort}\ 1).\ A \to \mathsf{List}\ A \to \mathsf{List}\ A.$$

The constructors take A as an explicit first argument. For example, the one-element list [5] is cons Nat 5 (nil Nat). Within one list, A never changes: the tail of cons $A\ a\ as$ is again a List A. An argument of the type that stays the same in every constructor and every recursive argument is called a *parameter*. Because it never changes, the recursor can take it once, before the motive, and treat it as fixed:

$$\mathsf{List.rec}_v : \Pi(A{:}\mathsf{Sort}\ 1).\ \Pi(M{:}\mathsf{List}\ A \to \mathsf{Sort}\ v).\ M\ (\mathsf{nil}\ A) \to$$
$$(\Pi(a{:}A).\ \Pi(as{:}\mathsf{List}\ A).\ M\ as \to M\ (\mathsf{cons}\ A\ a\ as)) \to \Pi(x{:}\mathsf{List}\ A).\ M\ x$$

$$\mathsf{List.rec}_v\ A\ M\ m_1\ m_2\ (\mathsf{nil}\ A') \ \to_\iota\ m_1$$
$$\mathsf{List.rec}_v\ A\ M\ m_1\ m_2\ (\mathsf{cons}\ A'\ a\ as) \ \to_\iota\ m_2\ a\ as\ (\mathsf{List.rec}_v\ A\ M\ m_1\ m_2\ as)$$

Apart from the leading A, this is the recipe of §7.2 once more: cons has two arguments, of which only as is recursive, so its case gets one induction hypothesis. The rules ignore the parameter A' stored in the constructor; in a well-typed term it is convertible to A. For example, the length of a list is

$$\mathit{length} := \lambda(A{:}\mathsf{Sort}\ 1).\ \lambda(x{:}\mathsf{List}\ A).$$
$$\mathsf{List.rec}_1\ A\ (\lambda(_{:}\mathsf{List}\ A).\ \mathsf{Nat})\ \mathsf{zero}\ (\lambda(a{:}A).\ \lambda(as{:}\mathsf{List}\ A).\ \lambda(r{:}\mathsf{Nat}).\ \mathsf{succ}\ r)\ x.$$

Indices: vectors. A List Nat may have any length. What if we want the length to be part of the type, so that Vec Nat 0 and Vec Nat 2 are different types?

```
inductive Vec (A : Sort 1) : Nat -> Sort 1 where
  | vnil  : Vec A 0
  | vcons : (n : Nat) -> A -> Vec A n -> Vec A (succ n)
```

The element type A is still a parameter. The length is not: vnil has length 0, and vcons takes a vector of length n and returns one of length succ n. An argument of the type that may differ

between constructors, and between a constructor and its recursive arguments, is called an *index*. The constructors are

$$\begin{aligned}&\mathsf{vnil} : \Pi(A{:}\mathsf{Sort}\ 1).\ \mathsf{Vec}\ A\ \mathsf{zero},\\ &\mathsf{vcons} : \Pi(A{:}\mathsf{Sort}\ 1).\ \Pi(n{:}\mathsf{Nat}).\ A \to \mathsf{Vec}\ A\ n \to \mathsf{Vec}\ A\ (\mathsf{succ}\ n).\end{aligned}$$

Because an index varies, the recursor cannot fix it in advance. Instead, the motive takes it as an argument, $M : \Pi(n{:}\mathsf{Nat}).\ \mathsf{Vec}\ A\ n \to \mathsf{Sort}\ v$, and the case for vcons is

$$\Pi(n{:}\mathsf{Nat}).\ \Pi(a{:}A).\ \Pi(xs{:}\mathsf{Vec}\ A\ n).\ M\ n\ xs \to M\ (\mathsf{succ}\ n)\ (\mathsf{vcons}\ A\ n\ a\ xs).$$

The induction hypothesis is about a vector of length n, read off the type $\mathsf{Vec}\ A\ n$ of the recursive argument, and the conclusion is about one of length $\mathsf{succ}\ n$, read off the result type of the constructor. `examples/generic-vec.leanmini` declares this type and checks a small term that uses it.

7.5 The General Recipe

In Lean, a user never writes recursors or ι-rules. The user writes an `inductive` declaration, and the kernel checks it and generates the type, the constructors, the recursor, and the rules, following the recipe that §§7.2–7.4 applied by hand. This subsection states that procedure for a single inductive type with parameters and indices. Mutual and nested inductive types, which Lean also supports, are out of scope.

The procedure also answers a question that T-Const leaves open. A constant without a body is an assumption: T-Const would accept $c : \Pi(P{:}\mathsf{Sort}\ 0).\ P$ just as readily as $\mathsf{zero} : \mathsf{Nat}$, and with it the system would be inconsistent. The constants generated below are not arbitrary assumptions. They are instances of a fixed schema, the inductive types of the Calculus of Inductive Constructions, whose consistency is known [4].

Telescopes. We write $(\vec{x} : \vec{A})$ for a sequence of bindings $(x_1 : A_1) \cdots (x_k : A_k)$ in which each A_i may mention $x_1, \ldots, x_{i-1}$, $\Pi(\vec{x}{:}\vec{A}).\ B$ for $\Pi(x_1{:}A_1).\ \cdots\ \Pi(x_k{:}A_k).\ B$, and $f\ \vec{x}$ for $f\ x_1 \cdots x_k$. The length of $\vec{x}$ is written $|\vec{x}|$.

Declarations. A declaration introduces a type former I and constructors $c_1, \ldots, c_n$:

$$\begin{aligned}I &: \Pi(\vec{p}{:}\vec{P}).\ \Pi(\vec{\imath}{:}\vec{J}).\ \mathsf{Sort}\ u\\ c_j &: \Pi(\vec{p}{:}\vec{P}).\ \Pi(\vec{a}{:}\vec{A}_j).\ I\ \vec{p}\ \vec{t}_j \qquad (j = 1, \ldots, n)\end{aligned}$$

The *parameters* $\vec{p}$ are shared by all constructors and appear unchanged in every result type. The *indices* $\vec{\imath}$ may differ between constructors: the index terms $\vec{t}_j$ may mention $\vec{p}$ and the constructor's own arguments $\vec{a}$, which are called its *fields*. For the examples so far, with the leading Π over the parameters omitted from the constructor types:

Type former	Parameters	Indices	Constructors
$\mathsf{Nat} : \mathsf{Sort}\ 1$	none	none	$\mathsf{zero} : \mathsf{Nat}$,
			$\mathsf{succ} : \mathsf{Nat} \to \mathsf{Nat}$
$\mathsf{Tree} : \mathsf{Sort}\ 1$	none	none	leaf, node, branch
$\mathsf{List} : \mathsf{Sort}\ 1 \to \mathsf{Sort}\ 1$	$(A : \mathsf{Sort}\ 1)$	none	$\mathsf{nil} : \mathsf{List}\ A$,
			$\mathsf{cons} : A \to \mathsf{List}\ A \to \mathsf{List}\ A$
$\mathsf{Vec} : \mathsf{Sort}\ 1 \to \mathsf{Nat} \to \mathsf{Sort}\ 1$	$(A : \mathsf{Sort}\ 1)$	$(n : \mathsf{Nat})$	$\mathsf{vnil} : \mathsf{Vec}\ A\ \mathsf{zero}$,
			$\mathsf{vcons} : \Pi(n{:}\mathsf{Nat}).\ A \to \mathsf{Vec}\ A\ n \to \mathsf{Vec}\ A\ (\mathsf{succ}\ n)$

Acceptance. The kernel accepts a declaration only if the following conditions hold.

(1) The type of I is a type in the current signature.
(2) With I added to the signature, each constructor type is a type, and it ends in $I\ \vec{p}\ \vec{t}_j$, where $\vec{p}$ are exactly the parameters in order and I does not occur in $\vec{t}_j$.
(3) *Strict positivity.* Each field type either does not mention I, or has the form $\Pi(\vec{y}{:}\vec{B}).\, I\ \vec{p}\ \vec{s}$ where I occurs neither in $\vec{B}$ nor in $\vec{s}$. A field of the second form is *recursive*.
(4) *Universes.* If $u \geq 1$, each field type lies in some Sort w with $w \leq u$. If $u = 0$, there is no condition.

Condition 2 lets the recursor treat the parameters uniformly. Condition 3 rules out declarations such as

```
inductive Bad where
  | mk : (Bad -> False) -> Bad
```

whose field mentions `Bad` to the left of an arrow. Case analysis on `Bad` would let one define $f : \texttt{Bad} \to \texttt{False}$ by $f\ (\texttt{mk}\ g) = g\ (\texttt{mk}\ g)$. Then $f\ (\texttt{mk}\ f)$ is a closed term of type `False`, and it reduces to itself, so it has no normal form. For a type `Tree` under declaration, some field types and their status are:

Field type	Status	Reason
Nat	accepted, not recursive	does not mention `Tree`
Tree	accepted, recursive	induction hypothesis $M\ a_k$
Nat → Tree	accepted, recursive	produces trees; hypothesis $\Pi(n{:}\mathrm{Nat}).\, M\ (a_k\ n)$
Tree → Nat	rejected	`Tree` to the left of an arrow
List Tree	rejected here	nested occurrence; Lean accepts it, but this is out of scope

A field of type Nat → Tree is a tree with infinitely many children, one for each number; the recursor can still recurse into each child $a_k\ n$. A field of type Tree → Nat consumes trees instead of containing them, and it is the shape that makes `Bad` paradoxical. Condition 4 prevents a type from containing a larger universe. Without it, a declaration $T : \mathrm{Sort}\ 1$ with a constructor $\texttt{mk} : \mathrm{Sort}\ 1 \to T$ would place a copy of all of Sort 1 inside Sort 1, which is the self-containment that §10.2 shows to be inconsistent. For $u = 0$ no condition is needed, because Sort 0 is impredicative (§10.3).

The recursor. For an accepted declaration, the kernel generates a recursor $I.\mathrm{rec}_v$ for each admissible level v (see below):

$$I.\mathrm{rec}_v : \Pi(\vec{p}{:}\vec{P}).\ \Pi(M{:}(\Pi(\vec{i}{:}\vec{J}).\, I\ \vec{p}\ \vec{i} \to \mathrm{Sort}\ v)).\ \Pi(m_1{:}C_1).\ \cdots\ \Pi(m_n{:}C_n).\ \Pi(\vec{i}{:}\vec{J}).\, \Pi(x{:}I\ \vec{p}\ \vec{i}).\, M\ \vec{i}\ x.$$

The parameters come first and stay fixed, as for lists; the motive takes the indices and the element, as for vectors. The minor premise C_j asks for the case of constructor c_j: given its fields and an induction hypothesis for each recursive field, build M for c_j applied to those fields:

$$C_j = \Pi(\vec{a}{:}\vec{A}_j).\ \Pi(\vec{h}{:}\vec{H}_j).\ M\ \vec{t}_j\ (c_j\ \vec{p}\ \vec{a}),$$
$$H_{j,k} = \Pi(\vec{y}{:}\vec{B}).\, M\ \vec{s}\ (a_k\ \vec{y}) \quad \text{for each recursive field } a_k : \Pi(\vec{y}{:}\vec{B}).\, I\ \vec{p}\ \vec{s}.$$

The final arguments are the indices and the element x itself, the major premise.

The rules. For each constructor, the kernel generates one ι-rule, which applies the minor premise to the fields and to the induction hypotheses, computed by recursive calls:

$$I.\mathrm{rec}_v\ \vec{p}\ M\ \vec{m}\ \vec{\imath}\ (c_j\ \vec{p}\,'\ \vec{a}) \ \to_\iota\ m_j\ \vec{a}\ \vec{h}, \qquad h_k = \lambda(\vec{y}{:}\vec{B}).\, I.\mathrm{rec}_v\ \vec{p}\ M\ \vec{m}\ \vec{s}\ (a_k\ \vec{y}).$$

As data for Algorithm 2 (§9.5), the major premise is at position $|\vec{p}| + n + |\vec{\imath}| + 2$, the constructor has $|\vec{p}|$ parameters and $|\vec{a}|$ fields, and the right-hand side is $\lambda\vec{p}\,M\,\vec{m}\,\vec{\imath}.\ \lambda\vec{a}.\ m_j\ \vec{a}\ \vec{h}$. In a well-typed redex,

$\vec{p}\,'$ is convertible to $\vec{p}$, so the rule ignores it, as the rules for lists ignore A'. For Nat, Tree, List, and Vec, the procedure produces exactly the recursors and rules of §§7.2–7.4, and the recursor types agree with the ones Lean 4 generates, up to the names of bound variables and implicit arguments.

Eliminating from propositions. If $u \geq 1$, the recursor is generated for every level v. If $u = 0$, so that I is a proposition, the recursor is generated only for $v = 0$, unless I is a *subsingleton*: it has at most one constructor, and each field either is a proof (its type lies in Sort 0) or occurs among the index terms $\vec{t}_j$. The restriction keeps data from depending on which proof of a proposition was given. For example, $P \vee Q$ has two constructors, so a function from proofs of $P \vee Q$ into Nat could return 0 for one and 1 for the other. Lean's `Or.rec` and `Exists.rec` therefore eliminate only into Prop, while `And.rec` and `Eq.rec` eliminate into every universe: `And` has one constructor whose fields are proofs, and `Eq` (§7.6) has one constructor with no fields.

So the kernel's rules stay fixed while the signature grows: one generic procedure turns each declaration into signature entries, and a single reduction function runs their rules (§9.5).

7.6 Equality Is an Inductive Type Too

To state facts about numbers, such as $0 + n = n$, we need a way to say that two things are equal. One might expect equality to be a primitive of the kernel. Surprisingly, it is just another inductive type:

```
inductive Eq {A : Sort u} (a : A) : A -> Prop where
  | refl : Eq a a
```

For $a, b : A$, the type $\mathsf{Eq}_u\ A\ a\ b$ is the proposition that a and b are equal; when A is Nat we write it $a = b$. In the terms of §7.5, A and a are parameters and b is an index:

$$\mathsf{Eq}_u : \Pi(A{:}\mathrm{Sort}\ u).\ A \to A \to \mathrm{Sort}\ 0, \qquad \mathsf{refl}_u : \Pi(A{:}\mathrm{Sort}\ u).\ \Pi(a{:}A).\ \mathsf{Eq}_u\ A\ a\ a.$$

The only constructor refl_u has no fields, and its index term is a: it can only prove equations whose two sides are the same. So Eq is an *inductive family* like Vec, not a single type.

The general recipe gives its recursor. Because b is an index, the motive takes b and the proof $h : \mathsf{Eq}_u\ A\ a\ b$, and there is one minor premise, for refl_u, with no fields and no hypotheses:

$$\begin{aligned}\mathsf{Eq.rec}_{u,v} : {}& \Pi(A{:}\mathrm{Sort}\ u).\ \Pi(a{:}A).\ \Pi(M{:}(\Pi(b{:}A).\ \mathsf{Eq}_u\ A\ a\ b \to \mathrm{Sort}\ v)).\\ & M\ a\ (\mathsf{refl}_u\ A\ a) \to \Pi(b{:}A).\ \Pi(h{:}\mathsf{Eq}_u\ A\ a\ b).\ M\ b\ h\end{aligned}$$

$$\mathsf{Eq.rec}_{u,v}\ A\ a\ M\ m\ b\ (\mathsf{refl}_u\ A'\ c)\ \to_\iota\ m.$$

Eq is a subsingleton, so v is arbitrary. Read logically, the recursor says: if something holds in the case where the two sides are identical, then it holds whenever we have a proof that they are equal. Most uses need only the non-dependent version $\mathsf{Eq.ndrec}_{u,v}$, whose motive ignores the proof. It is a definition built from $\mathsf{Eq.rec}_{u,v}$,

$$\begin{aligned}\mathsf{Eq.ndrec}_{u,v} := {}& \lambda(A{:}\mathrm{Sort}\ u).\ \lambda(a{:}A).\ \lambda(M{:}A \to \mathrm{Sort}\ v).\ \lambda(m{:}M\ a).\ \lambda(b{:}A).\ \lambda(h{:}\mathsf{Eq}_u\ A\ a\ b).\\ & \mathsf{Eq.rec}_{u,v}\ A\ a\ (\lambda(b'{:}A).\ \lambda(_{:}\mathsf{Eq}_u\ A\ a\ b').\ M\ b')\ m\ b\ h,\end{aligned}$$

and Fig. 4 summarizes the constants. $\mathsf{Eq.ndrec}_{u,v}$ is Leibniz's principle: if $a = b$, then whatever holds of a holds of b. It *transports* $m : M\ a$ along a proof of $a = b$ to $M\ b$, and its rule says that transporting along refl does nothing. In a well-typed redex, b and c are both convertible to a, so the rules can ignore them.

For example, suppose $P : \mathsf{Nat} \to \mathrm{Sort}\ 0$, $a, b : \mathsf{Nat}$, $h : a = b$, and $pa : P\ a$. Choosing the motive $M = P$ gives

$$\mathsf{Eq.ndrec}_{1,0}\ \mathsf{Nat}\ a\ P\ pa\ b\ h\ :\ P\ b.$$

This one line is the logical content of rewriting: a fact about a becomes a fact about b. The `rw` tactic of §8 builds such terms automatically.

Constant and type		Lean
Eq_u	$: \Pi(A{:}\mathrm{Sort}\ u).\ A \to A \to \mathrm{Sort}\ 0$	`Eq`
refl_u	$: \Pi(A{:}\mathrm{Sort}\ u).\ \Pi(a{:}A).\ \mathsf{Eq}_u\ A\ a\ a$	`Eq.refl`
$\mathsf{Eq.rec}_{u,v}$	$: \Pi(A{:}\mathrm{Sort}\ u).\ \Pi(a{:}A).\ \Pi(M{:}(\Pi(b{:}A).\ \mathsf{Eq}_u\ A\ a\ b \to \mathrm{Sort}\ v)).$ $M\ a\ (\mathsf{refl}_u\ A\ a) \to \Pi(b{:}A).\ \Pi(h{:}\mathsf{Eq}_u\ A\ a\ b).\ M\ b\ h$	`Eq.rec`
$\mathsf{Eq.ndrec}_{u,v}$	$: \Pi(A{:}\mathrm{Sort}\ u).\ \Pi(a{:}A).\ \Pi(M{:}A \to \mathrm{Sort}\ v).$ $M\ a \to \Pi(b{:}A).\ \mathsf{Eq}_u\ A\ a\ b \to M\ b$	`Eq.ndrec` (definition)

$$\mathsf{Eq.rec}_{u,v}\ A\ a\ M\ m\ b\ (\mathsf{refl}_u\ A\ c) \ \to_\iota\ m$$
$$\mathsf{Eq.ndrec}_{u,v}\ A\ a\ M\ m\ b\ (\mathsf{refl}_u\ A\ c) \to_{\delta\beta\iota} m$$

Fig. 4. The constants for equality and their computation rules. The rule for $\mathsf{Eq.ndrec}_{u,v}$ is derived from its definition by δ, β, and the rule for $\mathsf{Eq.rec}_{u,v}$.

In Lean, the type argument of `Eq` is *implicit*, written `{A}` above, and the elaborator fills it in. Our kernel sees only fully explicit terms, so we write every argument; filling them in is the job of the layer above the kernel (§8).

Definitional versus propositional equality. There are now two notions of equality. Definitional equality $\equiv$ is decided by the kernel by computation; propositional equality `Eq` is a type, whose proofs may require reasoning. The constructor refl connects them: by T-Conv, $\mathsf{refl}_u\ A\ a : \mathsf{Eq}_u\ A\ a\ b$ whenever $a \equiv b$. For example, with the definition of add from §7.2 and s its step,

$$n + 0 \ \to_{\delta\beta}\ \mathsf{Nat.rec}_1\ (\lambda(_{:}\mathrm{Nat}).\ \mathrm{Nat})\ n\ s\ \mathrm{zero} \ \to_\iota\ n,$$

so $\mathsf{refl}_1\ \mathrm{Nat}\ n : n + 0 = n$, and Lean proves this by `rfl`. In contrast,

$$0 + n \ \to_{\delta\beta}\ \mathsf{Nat.rec}_1\ (\lambda(_{:}\mathrm{Nat}).\ \mathrm{Nat})\ 0\ s\ n$$

is stuck: n is a variable, and the ι-rules fire only on zero and succ. So $0 + n = n$ is not provable by refl and needs induction.

7.7 Example: $0 + n = n$

We prove $\Pi(n{:}\mathrm{Nat}).\ 0 + n = n$ by induction, using $\mathsf{Nat.rec}_0$ with the motive $M_0 = \lambda(n{:}\mathrm{Nat}).\ 0 + n = n$.

- *Base.* We need $M_0\ 0$, which is convertible to $0 + 0 = 0$. Since $0 + 0$ computes to 0, the term $\mathsf{refl}_1\ \mathrm{Nat}\ 0$ works.
- *Step.* Given $k : \mathrm{Nat}$ and $ih : 0 + k = k$, we need $M_0\ (\mathrm{succ}\ k)$, that is, $0 + \mathrm{succ}\ k = \mathrm{succ}\ k$. By the second ι-rule, $0 + \mathrm{succ}\ k$ computes to $\mathrm{succ}\ (0 + k)$, so it suffices to prove $\mathrm{succ}\ (0 + k) = \mathrm{succ}\ k$. We start from $\mathsf{refl}_1\ \mathrm{Nat}\ (\mathrm{succ}\ (0 + k))$ and transport it along ih with the motive $N = \lambda(b{:}\mathrm{Nat}).\ \mathrm{succ}\ (0 + k) = \mathrm{succ}\ b$, which replaces the second $0 + k$ by k:

$$step = \lambda(k{:}\mathrm{Nat}).\ \lambda(ih{:}0 + k = k).\ \mathsf{Eq.ndrec}_{1,0}\ \mathrm{Nat}\ (0 + k)\ N\ (\mathsf{refl}_1\ \mathrm{Nat}\ (\mathrm{succ}\ (0 + k)))\ k\ ih.$$

The proof is

$$zero_add = \mathsf{Nat.rec}_0\ M_0\ (\mathsf{refl}_1\ \mathrm{Nat}\ 0)\ step\ :\ \Pi(n{:}\mathrm{Nat}).\ 0 + n = n.$$

The kernel checks it with Algorithm 1. Each argument is checked against the corresponding part of the type of $\mathsf{Nat.rec}_0$, instantiated with M_0, and every comparison succeeds after normalization. Writing such terms by hand is tedious, and finding the right motive is the hard part; tactics (§8) automate both.

7.8 Changes to the Kernel

Infer gains a case for constants, which looks up the type in Σ (T-Const). Check is unchanged; only Whnf, and through it IsDefEq, learns the new δ- and ι-steps. §9.5 shows how a single Whnf handles every inductive type, and §11 discusses what inductive types change in the metatheory.

8 TACTICS: BUILDING PROOF TERMS INTERACTIVELY

Lean users rarely write terms like *zero_add* by hand. They write a tactic script instead:

```
theorem zero_add (n : Nat) : 0 + n = n := by
  induction n with
  | zero => rfl
  | succ k ih => rw [Nat.add_succ, ih]
```

Each tactic is a small program that builds part of the proof term. Tactics run outside the kernel. Their output is an ordinary term, which the kernel checks with Algorithm 1 like any other term. This section shows how the common tactics construct terms in our system.

8.1 Goals and Proof States

A *goal* $\langle \Gamma \vdash ?_m : T \rangle$ asks for a term of type T in the context Γ. The *hole* $?_m$ stands for the term still to be found; Lean calls it a *metavariable*. A *proof state* is a list of goals together with a *partial proof term*, a term that may contain holes.

To prove T, we start from the partial term $?_0$ and the single goal $\langle \cdot \vdash ?_0 : T \rangle$. A tactic works on the first goal $\langle \Gamma \vdash ?_m : T \rangle$. It *assigns* to $?_m$ a term that may contain fresh holes, and replaces the goal by one new goal for each fresh hole. When no goals remain, the partial term contains no holes, and the kernel checks it against T. A hole may sit under binders, as $?_1$ does in $\lambda(x{:}A).\, ?_1$, and the term that fills it may use those bound variables. That is why each goal carries its own context.

Each tactic must make sure that its assignment has type T in Γ, assuming each fresh hole is later filled with a term of the type its goal states. We write $\langle \Gamma \vdash ?_m : T \rangle \rightsquigarrow$ new goals, followed by the assignment.

A first example. The script of `examples/identity.leanmini` proves the opening example of §1:

```
goal forall (P : Sort 0), P -> P
intro P
intro p
exact p
qed
```

The proof state evolves as follows; each line shows the goals and the assignment made by the tactic.

Tactic	Goals afterwards	Assignment
(start)	$\langle \cdot \vdash ?_0 : \Pi(P{:}\mathrm{Sort}\ 0).\, P \to P \rangle$	
`intro P`	$\langle P : \mathrm{Sort}\ 0 \vdash ?_1 : P \to P \rangle$	$?_0 := \lambda(P{:}\mathrm{Sort}\ 0).\, ?_1$
`intro p`	$\langle P : \mathrm{Sort}\ 0, p : P \vdash ?_2 : P \rangle$	$?_1 := \lambda(p{:}P).\, ?_2$
`exact p`	none	$?_2 := p$

Substituting the assignments into $?_0$ gives $\lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(p{:}P).\, p$, the term we wrote by hand, and `qed` sends it to Check. A list of goals alone would not be enough: after `intro p`, the goal P does not remember that its solution must be wrapped in two λs. The partial term records exactly that.

8.2 Basic Tactics

`intro x`. If T normalizes to a Π type, the tactic moves its argument into the context:

$$\langle \Gamma \vdash ?_m : \Pi(y{:}A).\, B \rangle \;\rightsquigarrow\; \langle \Gamma, x : A \vdash ?_1 : B[y \mapsto x] \rangle, \qquad ?_m := \lambda(x{:}A).\, ?_1.$$

The assignment has the right type by T-Lam. The name x must not be in $\mathrm{dom}(\Gamma)$; otherwise the new assumption would shadow an existing one, which makes the kernel unsound (§9.2), so the tactic must rename it.

`exact t`. The tactic closes the goal with a given term:

$$\langle \Gamma \vdash ?_m : T \rangle \;\rightsquigarrow\; \text{no goals}, \qquad ?_m := t,$$

after running $\textsc{Check}(\Gamma, t, T)$ itself. The kernel would reject an ill-typed t at the end anyway; checking early only gives a better error message.

`apply f`. If f has type $A_1 \to \cdots \to A_n \to B$ with $B \equiv T$, the tactic leaves the arguments as new goals:

$$\langle \Gamma \vdash ?_m : T \rangle \;\rightsquigarrow\; \langle \Gamma \vdash ?_1 : A_1 \rangle, \ldots, \langle \Gamma \vdash ?_n : A_n \rangle, \qquad ?_m := f\ ?_1 \cdots ?_n.$$

The assignment has type B by n uses of T-App, and type T by T-Conv. When the type of f is a dependent Π, later A_i and B mention earlier arguments, and the tactic must find those arguments by matching B against T. Lean does this by *unification*, solving for metavariables. Our restricted version handles only non-dependent arrows.

For example, transitivity of implication (the second exercise of §6) works backwards from the conclusion:

```
goal forall (P : Sort 0), forall (Q : Sort 0), forall (R : Sort 0), (P -> Q) -> (Q -> R) -> P -> R
intro P
intro Q
intro R
intro f
intro g
intro p
apply g
apply f
exact p
qed
```

After the six `intro`s the goal is $\langle \Gamma \vdash ?_6 : R \rangle$ with $\Gamma = P, Q, R : \mathrm{Sort}\ 0,\ f : P \to Q,\ g : Q \to R,\ p : P$. No hypothesis has type R, so `exact` cannot close it, but g ends in R: `apply g` assigns $?_6 := g\ ?_7$ and leaves $?_7 : Q$. Likewise `apply f` assigns $?_7 := f\ ?_8$ and leaves $?_8 : P$, which `exact p` closes. The body of the resulting term is $g\ (f\ p)$. Note the difference between the two tactics: `exact g` fails, because g has type $Q \to R$, not R, whereas `apply g` turns the missing argument of g into a new goal.

`change T'`.. The tactic replaces the goal by a definitionally equal one:

$$\langle \Gamma \vdash ?_m : T \rangle \;\rightsquigarrow\; \langle \Gamma \vdash ?_1 : T' \rangle, \qquad ?_m := ?_1,$$

provided T' is a type and $T' \equiv T$. It adds nothing to the term; it only changes how the goal is displayed, and T-Conv justifies it. `change` is needed because `rw` (below) finds subterms only up to α-equivalence: a subterm that appears only after computation must first be exposed with `change`, as in the example of §8.5.

`rfl`. If T normalizes to $\mathsf{Eq}_u\ A\ a\ b$ with $a \equiv b$, the tactic closes the goal:

$$\langle \Gamma \vdash ?_m : \mathsf{Eq}_u\ A\ a\ b \rangle \ \rightsquigarrow\ \text{no goals}, \qquad ?_m := \mathsf{refl}_u\ A\ a.$$

The assignment has type $\mathsf{Eq}_u\ A\ a\ a$, which is convertible to the goal. So `rfl` proves exactly the equations that hold by definitional equality (§7.6).

8.3 Induction

Let $n : \mathsf{Nat}$ be a variable in Γ, and let $\Gamma \vdash T : \mathsf{Sort}\ u$. The tactic `induction n` abstracts n in the goal to form the motive, and produces one goal per constructor:

$$\langle \Gamma \vdash ?_m : T \rangle \ \rightsquigarrow\ \langle \Gamma \vdash ?_z : T[n \mapsto \mathsf{zero}] \rangle,\ \langle \Gamma, k : \mathsf{Nat}, ih : T[n \mapsto k] \vdash ?_s : T[n \mapsto \mathsf{succ}\ k] \rangle,$$
$$?_m := \mathsf{Nat.rec}_u\ (\lambda(z{:}\mathsf{Nat}).\, T[n \mapsto z])\ ?_z\ (\lambda(k{:}\mathsf{Nat}).\, \lambda(ih{:}T[n \mapsto k]).\, ?_s)\ n.$$

With the motive $M = \lambda(z{:}\mathsf{Nat}).\, T[n \mapsto z]$, the type of $\mathsf{Nat.rec}_u\ M$ asks for $M\ \mathsf{zero}$ and $\Pi(k{:}\mathsf{Nat}).\, M\ k \to M\ (\mathsf{succ}\ k)$, and the result has type $M\ n$. These are convertible to the types of the two goals and to T, by β. The construction is correct only if the names are fresh, the level u is right, and no other hypothesis mentions n:

- The motive binds a fresh name z, not n: since n is already in Γ, writing $\lambda(n{:}\mathsf{Nat}).\, T$ would shadow it, which is the pitfall of §9.2. Likewise k and ih must be fresh.
- The level u of the motive comes from the type of the goal, found by InferSort. For a proposition it is 0, and $\mathsf{Nat.rec}_0$ is induction.
- If another hypothesis mentions n, for example $h : n = 2$, the new goals still contain h about the original n, and ih says nothing about it. The goals are then correct but may be unprovable. Lean's `induction` first moves such hypotheses into the goal (it *reverts* them), so that they are part of the motive. Our version requires that no other hypothesis mentions n.

8.4 Rewriting

Let $h : \mathsf{Eq}_u\ A\ a\ b$, and let the goal T be a proposition. The tactic `rw [h]` replaces a by b in the goal in four steps.

(1) *Abstract.* Choose a fresh x and let C be T with every occurrence of a replaced by x. Occurrences are found up to α-equivalence, not up to conversion, and an occurrence under a binder that binds a free variable of a is skipped, since there it denotes something else.
(2) *Check the motive.* Check that $\Gamma, x : A \vdash C : \mathsf{Sort}\ 0$. This can fail, as shown below.
(3) *Build the term.* The new goal is $C[x \mapsto b]$, the goal with a replaced by b:

$$\langle \Gamma \vdash ?_m : T \rangle \ \rightsquigarrow\ \langle \Gamma \vdash ?_1 : C[x \mapsto b] \rangle,$$
$$?_m := \mathsf{Eq.ndrec}_{u,0}\ A\ a\ (\lambda(x{:}A).\, C \to T)\ (\lambda(p{:}T).\, p)\ b\ h\ ?_1.$$

(4) *Try* `rfl`. Like Lean, close the new goal by `rfl` if possible.

To see that the assignment has type T, write $N = \lambda(x{:}A).\, C \to T$ for the motive. Since $C[x \mapsto a] = T$, the term $N\ a$ is convertible to $T \to T$, which the identity $\lambda(p{:}T).\, p$ inhabits. Transport along h then gives $N\ b$, which is convertible to $C[x \mapsto b] \to T$, and applying it to $?_1$ gives T. Lean builds a different term, `Eq.mpr (congrArg (fun x => C) h)`, from the same idea.

When the motive is ill typed. Abstracting a can break the goal. Let $\Gamma = n : \mathsf{Nat},\ h : n = 2,\ q : n = 0$, and suppose we run `rw [h]` on the goal $\mathsf{Eq}_0\ (n = 0)\ q\ q$, a statement about the proof q. Abstracting n gives $C = \mathsf{Eq}_0\ (x = 0)\ q\ q$, which is ill typed: q has type $n = 0$, not $x = 0$, and q itself cannot be abstracted. Step 2 fails, and Lean reports the same situation as "motive is not type correct". Without step 2, the tactic would produce a term that the kernel rejects.

8.5 Example: $0 + n = n$ with Tactics

In our system the proof of §7.7 becomes the following script, shown with the notation of this paper; the runnable version, which writes add zero n for $0 + n$, is `examples/zero-add.leanmini`.

```
intro n
induction n
rfl
change succ (0 + k) = succ k
rw [ih]
```

The proof state evolves as follows.

(1) Initially there is one goal, $\langle \cdot \vdash ?_0 : \Pi(n{:}\mathsf{Nat}).\, 0 + n = n \rangle$.
(2) `intro n` assigns $?_0 := \lambda(n{:}\mathsf{Nat}).\, ?_1$ and leaves $\langle n : \mathsf{Nat} \vdash ?_1 : 0 + n = n \rangle$.
(3) `induction n` assigns

$$?_1 := \mathsf{Nat.rec}_0\ (\lambda(z{:}\mathsf{Nat}).\, 0 + z = z)\ ?_2\ (\lambda(k{:}\mathsf{Nat}).\, \lambda(ih{:}0 + k = k).\, ?_3)\ n$$

and leaves two goals: $\langle n : \mathsf{Nat} \vdash ?_2 : 0 + 0 = 0 \rangle$ and $\langle n : \mathsf{Nat}, k : \mathsf{Nat}, ih : 0 + k = k \vdash ?_3 : 0 + \mathsf{succ}\ k = \mathsf{succ}\ k \rangle$.
(4) `rfl` closes the first goal with $?_2 := \mathsf{refl}_1\ \mathsf{Nat}\ (0 + 0)$, since $0 + 0 \equiv 0$.
(5) In the second goal, $0 + k$ does not occur syntactically, so `rw [ih]` would find nothing. `change` first exposes it, using $0 + \mathsf{succ}\ k \equiv \mathsf{succ}\ (0 + k)$: it assigns $?_3 := ?_4$ with the goal $?_4 : \mathsf{succ}\ (0 + k) = \mathsf{succ}\ k$. In Lean, rewriting with the lemma `Nat.add_succ`, which states $n + \mathsf{succ}\ m = \mathsf{succ}\ (n + m)$, produces the same goal.
(6) `rw [ih]` abstracts $0 + k$ to get $C = (\mathsf{succ}\ x = \mathsf{succ}\ k)$. With $T = (\mathsf{succ}\ (0 + k) = \mathsf{succ}\ k)$ for the current goal, it assigns

$$?_4 := \mathsf{Eq.ndrec}_{1,0}\ \mathsf{Nat}\ (0 + k)\ (\lambda(x{:}\mathsf{Nat}).\, C \to T)\ (\lambda(p{:}T).\, p)\ k\ ih\ ?_5.$$

The new goal $?_5 : \mathsf{succ}\ k = \mathsf{succ}\ k$ is closed at once by `rfl`, with $?_5 := \mathsf{refl}_1\ \mathsf{Nat}\ (\mathsf{succ}\ k)$.

No goals remain. Substituting the assignments into $?_0$ gives a closed term, and the kernel runs $\textsc{Check}(\cdot, t, \Pi(n{:}\mathsf{Nat}).\, 0 + n = n)$ on it. The term differs from the hand-written one of §7.7 in two places: it starts with $\lambda(n{:}\mathsf{Nat}).\ \ldots\ n$ because of `intro`, and its step uses the motive built by `rw`. Both are proofs of the same proposition, and the kernel does not care which one it receives.

8.6 Why Tactics Need Not Be Trusted

The kernel checks the final term, so tactics do not have to be correct for the system to be sound. A buggy tactic can fail, or it can produce an ill-typed term, which the kernel rejects; it cannot make the kernel accept a false theorem. The checks that tactics perform themselves, such as Check in `exact` or the motive check in `rw`, exist only to report errors early and clearly. The trusted part of the system is therefore small: Algorithm 1, the reduction and comparison functions of §9, and the signature of §7.

Lean's tactic framework is far richer than the one sketched here. It has metavariables solved by unification, an elaborator that fills in implicit arguments and expands notation such as `0 + n`, an `induction` that reverts dependent hypotheses, and automation such as `simp`. All of this lives on the untrusted side: Lean's kernel, like ours, only ever sees complete terms [9].

9 UNDER THE HOOD: SUBSTITUTION, α-EQUIVALENCE, AND CONVERSION

The previous sections treated three operations on terms as black boxes: substitution $B[x \mapsto a]$, used by T-App, by Freshen, and by every computation step; Whnf, used by Infer to expose a Π or a Sort; and IsDefEq, used by Check to decide $A \equiv B$. These operations are part of the trusted kernel, and

most kernel bugs hide in them, because they must handle the names of bound variables correctly. This section opens the boxes. §9.1 defines substitution, and §9.2 shows the same naming problem in contexts. §9.3 compares terms up to the names of bound variables. §9.4 defines computation and definitional equality, and §9.5–9.6 give the algorithms.

9.1 Free Variables and Substitution

Π and λ *bind* their variable: in $\lambda(x{:}A).\,t$, occurrences of x inside t refer to the argument. The binder's scope is the body only; x is not bound in the annotation A. A variable occurrence that is not bound by an enclosing Π or λ is *free*.

Definition 1 (Free variables).

$$\begin{aligned} \mathrm{FV}(\mathrm{Sort}\ u) &= \emptyset & \mathrm{FV}(\Pi(x{:}A).\,B) &= \mathrm{FV}(A) \cup (\mathrm{FV}(B) \setminus \{x\}) \\ \mathrm{FV}(x) &= \{x\} & \mathrm{FV}(\lambda(x{:}A).\,t) &= \mathrm{FV}(A) \cup (\mathrm{FV}(t) \setminus \{x\}) \\ \mathrm{FV}(t\ s) &= \mathrm{FV}(t) \cup \mathrm{FV}(s) \end{aligned}$$

For example, $\mathrm{FV}(\lambda(x{:}P).\,f\ x\ y) = \{P, f, y\}$, and $\mathrm{FV}(\Pi(P{:}\mathrm{Sort}\ 0).\,P \to P) = \emptyset$, so the latter is a closed term. Constants (§7.1) are never bound, so $\mathrm{FV}(c) = \emptyset$.

We write $t[x \mapsto s]$ for the result of replacing the free occurrences of x in t with s. The only subtle case is a binder. Consider $(\lambda(y{:}A).\,x)[x \mapsto y]$. The body says "ignore the argument and return the outer x". Replacing x by y naively gives $\lambda(y{:}A).\,y$, the identity function, which means something else: the free y has been *captured* by the binder. Substitution must avoid this by renaming the binder first.

Definition 2 (Capture-avoiding substitution). *For $\square \in \{\Pi, \lambda\}$:*

$$\begin{aligned} x[x \mapsto s] &= s \\ y[x \mapsto s] &= y && (y \neq x) \\ \mathrm{Sort}\ u[x \mapsto s] &= \mathrm{Sort}\ u \\ (t_1\ t_2)[x \mapsto s] &= t_1[x \mapsto s]\ \ t_2[x \mapsto s] \\ (\square(x{:}A).\,B)[x \mapsto s] &= \square(x{:}A[x \mapsto s]).\,B && \textit{(we say that } x \textit{ is shadowed)} \\ (\square(y{:}A).\,B)[x \mapsto s] &= \square(y{:}A[x \mapsto s]).\,B[x \mapsto s] && (y \neq x,\ y \notin \mathrm{FV}(s)) \\ (\square(y{:}A).\,B)[x \mapsto s] &= \square(z{:}A[x \mapsto s]).\,(B[y \mapsto z])[x \mapsto s] && (y \neq x,\ y \in \mathrm{FV}(s),\ z \textit{ fresh}) \end{aligned}$$

Here "z fresh" means $z \notin \mathrm{FV}(s) \cup \mathrm{FV}(B) \cup \{x\}$.

The following examples exercise each binder case.

$(\Pi(h{:}P).\,P)[P \mapsto Q]$	$= \Pi(h{:}Q).\,Q$	no capture possible; the T-App step of §6
$(\lambda(x{:}A).\,x)[x \mapsto s]$	$= \lambda(x{:}A).\,x$	x is shadowed; the body is untouched
$(\lambda(y{:}x).\,y)[x \mapsto s]$	$= \lambda(y{:}s).\,y$	the annotation is outside the scope of y
$(\lambda(y{:}A).\,x\ y)[x \mapsto f\ y]$	$= \lambda(z{:}A).\,f\ y\ z$	$y \in \mathrm{FV}(f\ y)$, so the binder is renamed to z

In the last example, the free y of the replacement still refers to the outer y, and the bound variable, now z, still refers to the argument. Without the renaming, the result would be $\lambda(y{:}A).\,f\ y\ y$, which passes its argument twice.

9.2 Variable Capture in Contexts

It is tempting to drop the side condition $x \notin \mathrm{dom}(\Gamma)$ of C-Ext and let a new assumption simply shadow an old one, as most programming languages do. Doing so makes the kernel unsound.

Consider

$$t = \lambda(x{:}\text{Sort } 0).\, \lambda(y{:}x).\, \lambda(x{:}\text{Sort } 0).\, y.$$

The variable y has the *outer* x as its type, and the innermost λ ignores its argument. So the correct type of t is

$$\Pi(x{:}\text{Sort } 0).\, \Pi(y{:}x).\, \Pi(x'{:}\text{Sort } 0).\, x.$$

A checker without Freshen looks up y in the context $x : \text{Sort } 0, y : x, x : \text{Sort } 0$ and returns its type, the bare name x. At that point x refers to the inner binder, and the checker returns

$$\Pi(x{:}\text{Sort } 0).\, \Pi(y{:}x).\, \Pi(x{:}\text{Sort } 0).\, x,$$

which says "from any element of any type, produce an element of every type". Given $A : \text{Sort } 0$ and $a : A$, the term $t\ A\ a$ then has type $\Pi(B{:}\text{Sort } 0).\, B$, a proof of False. Such A and a exist even in the empty context: $A = \Pi(P{:}\text{Sort } 0).\, P \to P$ is itself a proposition (§10.3), and the identity function $a = \lambda(P{:}\text{Sort } 0).\, \lambda(h{:}P).\, h$ proves it.

With Freshen, the innermost binder is renamed to x' before it enters the context, so the context becomes $x : \text{Sort } 0, y : x, x' : \text{Sort } 0$, the lookup of y returns x, which still refers to the outer binder, and Infer returns the correct type above.

The problem is the same as in substitution (§9.1): a name is moved into a scope where it means something else. It affects any representation that identifies variables by name, whether the context is a list or a hash map. There are three common remedies:

(1) Reject shadowing outright. This is the simplest option, but the reserved name _ used by $A \to B$ must be exempt and never added to the context.
(2) Rename on entry, as Freshen does.
(3) Represent bound variables by position (de Bruijn indices [6]) or use the *locally nameless* style, in which bound variables are indices and free variables carry unique identifiers. Lean 4 takes the latter approach.

A test suite should include t above and check that the kernel either rejects it or assigns it the correct type.

9.3 α-Equivalence

The names of bound variables carry no meaning. For example, $\lambda(x{:}A).\, x$ and $\lambda(y{:}A).\, y$ are the same function, and $\Pi(P{:}\text{Sort } 0).\, P \to P$ and $\Pi(Q{:}\text{Sort } 0).\, Q \to Q$ are the same proposition. Terms that differ only in the names of bound variables are called *α-equivalent*. A kernel cannot compare types with plain syntactic equality, which would reject the second pair.

Definition 3 (α-equivalence). *$=_\alpha$ is the smallest congruence on terms containing, for $\square \in \{\Pi, \lambda\}$,*

$$\square(x{:}A).\, B \;=_\alpha\; \square(y{:}A).\, B[x \mapsto y] \qquad \textit{whenever } y \notin \mathrm{FV}(B).$$

This definition is easy to state but not directly executable, since it does not say which names to choose. An implementation instead walks both terms in lockstep and remembers, for each side, the list of binders passed so far. A bound variable is then identified by *which* binder it refers to, not by its name.

To describe this walk, we use a judgment of the form $E_1; E_2 \vdash s =_\alpha t$, where E_1 and E_2 are lists of names. The list E_1 records the binders that enclose s, from the outermost to the innermost, and E_2 records those that enclose t. The judgment reads: "s, placed inside the binders E_1, and t, placed inside the binders E_2, are the same term up to the names of bound variables". Two lists are needed

because the two terms may use different names for corresponding binders. For example, comparing $\lambda(x{:}A).\,\lambda(y{:}B).\,x$ with $\lambda(a{:}A).\,\lambda(b{:}B).\,a$ passes two binders on each side and arrives at

$$x, y;\ a, b \vdash x =_\alpha a,$$

which holds because x and a both refer to the outer of the two binders. Comparing two whole terms starts from empty lists, $\cdot;\cdot \vdash s =_\alpha t$. The same symbol $=_\alpha$ appears in Definition 3 and in the judgment; the prefix $E_1; E_2 \vdash$ distinguishes the judgment, and Theorem 2 below shows that the two agree.

To say "the same binder" precisely, let $idx(E, x)$ be the position of the last occurrence of x in E, counted from the right starting at 0, or undefined if $x \notin E$. In the example, $idx(x, y;\ x) = idx(a, b;\ a) = 1$. Taking the *last* occurrence handles shadowing: in $\lambda(x{:}A).\,\lambda(x{:}B).\,x$ the body refers to the inner x.

$$\frac{}{E_1; E_2 \vdash \mathrm{Sort}\ u =_\alpha \mathrm{Sort}\ u}\ (\alpha\text{-Sort}) \qquad \frac{idx(E_1, x) = idx(E_2, y) = i}{E_1; E_2 \vdash x =_\alpha y}\ (\alpha\text{-Bound}) \qquad \frac{x \notin E_1 \qquad x \notin E_2}{E_1; E_2 \vdash x =_\alpha x}\ (\alpha\text{-Free})$$

$$\frac{E_1; E_2 \vdash A =_\alpha A' \qquad E_1, x; E_2, y \vdash B =_\alpha B'}{E_1; E_2 \vdash \Pi(x{:}A).\,B =_\alpha \Pi(y{:}A').\,B'}\ (\alpha\text{-Pi}) \qquad \frac{E_1; E_2 \vdash A =_\alpha A' \qquad E_1, x; E_2, y \vdash t =_\alpha t'}{E_1; E_2 \vdash \lambda(x{:}A).\,t =_\alpha \lambda(y{:}A').\,t'}\ (\alpha\text{-Lam})$$

$$\frac{E_1; E_2 \vdash t_1 =_\alpha t_1' \qquad E_1; E_2 \vdash t_2 =_\alpha t_2'}{E_1; E_2 \vdash t_1\ t_2 =_\alpha t_1'\ t_2'}\ (\alpha\text{-App})$$

Fig. 5. Algorithmic α-equivalence. By Theorem 2, two terms s and t are α-equivalent iff $\cdot;\cdot \vdash s =_\alpha t$.

Free variables are compared by name (α-Free), and bound variables by position (α-Bound). A variable that is bound on one side and free on the other is never equal to anything, which is why there is no rule for that case. Because both sides descend through binders together, E_1 and E_2 always have the same length. So comparing positions counted from the right is the same as comparing positions counted from the left, and an implementation may store each list as a map from names to binding depths, provided it restores the previous entry when it leaves a binder. Using positions instead of names everywhere gives the *de Bruijn representation* [6], in which α-equivalent terms are syntactically identical.

Constants (§7.1) are compared by name, like free variables.

Examples. The first example of §6 ended by comparing the inferred type with the stated one, where $P \longrightarrow P$ abbreviates $\Pi(_{:}P).\,P$:

$\cdot;\cdot \vdash \Pi(P{:}\mathrm{Sort}\ 0).\,\Pi(h{:}P).\,P =_\alpha \Pi(P{:}\mathrm{Sort}\ 0).\,\Pi(_{:}P).\,P$	by α-Pi, needs
$\cdot;\cdot \vdash \mathrm{Sort}\ 0 =_\alpha \mathrm{Sort}\ 0$	holds by α-Sort
$P; P \vdash \Pi(h{:}P).\,P =_\alpha \Pi(_{:}P).\,P$	by α-Pi, needs
$P; P \vdash P =_\alpha P$	holds by α-Bound (index 0)
$P, h; P, _ \vdash P =_\alpha P$	holds by α-Bound (index 1)

The names h and _ differ, but neither is ever looked up, so they do not matter. The wrong proof of §6 fails at the last step instead:

$\cdot;\cdot \vdash \Pi(P{:}\mathrm{Sort}\ 0).\,\Pi(h{:}P).\,\mathrm{Sort}\ 0 =_\alpha \Pi(P{:}\mathrm{Sort}\ 0).\,\Pi(_{:}P).\,P$	by α-Pi, needs
$\cdot;\cdot \vdash \mathrm{Sort}\ 0 =_\alpha \mathrm{Sort}\ 0$	holds by α-Sort
$P; P \vdash \Pi(h{:}P).\,\mathrm{Sort}\ 0 =_\alpha \Pi(_{:}P).\,P$	by α-Pi, needs
$P; P \vdash P =_\alpha P$	holds by α-Bound (index 0)
$P, h; P, _ \vdash \mathrm{Sort}\ 0 =_\alpha P$	no rule applies

Positions also tell apart terms that a careless comparison of names would confuse. $\lambda(x{:}A).\,\lambda(y{:}A).\,x$ and $\lambda(a{:}A).\,\lambda(b{:}A).\,b$ are not α-equivalent: the bodies reach x, y; $a, b \vdash x =_\alpha b$, and idx gives 1 for x but 0 for b. Conversely, $\lambda(x{:}A).\,\lambda(x{:}A).\,x$ and $\lambda(a{:}A).\,\lambda(b{:}A).\,b$ are α-equivalent, because the last occurrence of x in x, x has index 0, the same as b in a, b.

The rules of Fig. 5 are a different definition from Definition 3, so we must check that they agree.

Theorem 2 (Correctness of algorithmic α-equivalence). *For all terms s and t, $s =_\alpha t$ in the sense of Definition 3 if and only if $\cdot\,;\cdot \vdash s =_\alpha t$ is derivable by the rules of Fig. 5.*

The idea of the proof is that both sides say the same thing: s and t have the same de Bruijn representation. For the algorithm this is almost immediate, since its rules compare positions exactly as de Bruijn indices do. For Definition 3, the key fact is that renaming a bound variable does not change the de Bruijn representation. Appendix B gives the full proof.

9.4 Reduction and Conversion

Applying a λ-abstraction to an argument is computed by substituting the argument for the bound variable:

$$(\lambda(x{:}A).\,t)\ s\ \longrightarrow_\beta\ t[x \mapsto s].$$

This step is called *β-reduction*, and a subterm of the form $(\lambda(x{:}A).\,t)\ s$ is called a *β-redex*. §7 added two more kinds of steps: δ-reduction, which unfolds a definition (§7.1), and ι-reduction, which runs a recursor on a constructor (Fig. 3 and §7.5). We allow each step anywhere inside a term, and write $\longrightarrow$ for a single step of any kind. A term in which no step applies is in *normal form*. For example, with $Id = \lambda(X{:}\mathrm{Sort}\ 0).\,X$,

$$\lambda(h{:}Id\ P).\,(\lambda(y{:}P).\,y)\ h\ \longrightarrow_\beta\ \lambda(h{:}P).\,(\lambda(y{:}P).\,y)\ h\ \longrightarrow_\beta\ \lambda(h{:}P).\,h,$$

and the last term is in normal form. Reducing the inner redex first leads to the same result.

Definitional equality (or *conversion*) identifies terms that compute to the same result, up to the names of bound variables.

Definition 4 (Conversion). *$s \equiv t$ is the smallest equivalence relation that contains $=_\alpha$ and $\longrightarrow$ and is closed under all term constructors.*

To decide $s \equiv t$, the kernel normalizes both terms and compares the results with $=_\alpha$. This procedure relies on two facts. First, reduction is confluent: different reduction orders cannot lead to different normal forms, so $s \equiv t$ holds exactly when the normal forms of s and t are α-equivalent. Second, every well-typed term *has* a normal form, so normalization terminates. The second fact holds for our system, but it is a deep theorem (strong normalization, §11) [5]. A practical kernel also caps the number of reduction steps, so that a bug cannot make it loop forever; our implementation stops after 20,000 steps.

9.5 Computation Rules as Data

Figs. 3 and 4 write each ι-rule as a separate equation, which suggests that every new inductive type needs new reduction code in the kernel. It does not. A kernel stores the ι-rules as data in the signature, next to the recursor, and a single piece of code applies them for every inductive type.

Each recursor entry records the position of its *major premise*, the argument whose constructor decides which rule applies, and a list of *rules*, one per constructor. A rule names the constructor, the number of its *fields* (the arguments that the constructor stores, as opposed to the type parameters it merely carries), and a right-hand side *rhs*, an ordinary term. Each constructor entry records how many leading arguments are parameters. For three of our recursors, the data is as follows; type annotations on λ are omitted.

Recursor	Major	Constructor	Params	Fields	*rhs*
$\mathsf{Nat.rec}_u$	4th	zero	0	0	$\lambda M\, z\, s.\ z$
		succ	0	1	$\lambda M\, z\, s\, n.\ s\ n\ (\mathsf{Nat.rec}_u\ M\ z\ s\ n)$
$\mathsf{Eq.rec}_{u,v}$	6th	refl_u	2	0	$\lambda A\, a\, M\, m\, b.\ m$
$\mathsf{Tree.rec}_v$	5th	leaf	0	0	$\lambda M\, m_1\, m_2\, m_3.\ m_1$
		node	0	1	$\lambda M\, m_1\, m_2\, m_3\, t.\ m_2\ t\ (\mathsf{Tree.rec}_v\ M\ m_1\ m_2\ m_3\ t)$
		branch	0	2	$\lambda M\, m_1\, m_2\, m_3\, l\, r.\ m_3\ l\ r\ (\mathsf{Tree.rec}_v\ M\ m_1\ m_2\ m_3\ l)\ (\cdots\ r)$

The *rhs* takes the arguments before the major premise, then the fields of the constructor. For example, $\mathsf{Nat.rec}_u\ M\ z\ s\ (\mathsf{succ}\ n)$ selects the second rule and becomes

$$(\lambda M\, z\, s\, n.\ s\ n\ (\mathsf{Nat.rec}_u\ M\ z\ s\ n))\ M\ z\ s\ n,$$

which β-reduces to the right-hand side of the second equation in Fig. 3. Likewise, the only rule of $\mathsf{Eq.rec}_{u,v}$ fires on the term $\mathsf{Eq.rec}_{u,v}\ A\ a\ M\ m\ b\ (\mathsf{refl}_u\ A'\ c)$. The constructor refl_u has two parameters, A' and c, and no fields, so the *rhs* receives just $A\ a\ M\ m\ b$ and returns m.

9.6 Weak Head Normal Form and Normalization

Algorithm 2 defines the three functions that Algorithm 1 left open. Both Whnf and Normalize view a term as a *spine* $h\ a_1 \cdots a_n$: a head h that is not an application, applied to $n \geq 0$ arguments. Since application associates to the left (§2), $f\ a\ b\ c$ is the nested term $((f\ a)\ b)\ c$; following the function side of each application down to f gives the head, and the arguments met on the way, read from the inside out, are a, b, c. Every term has exactly one such decomposition; a term that is not an application is a spine with $n = 0$.

Algorithm 2 Weak head normalization with β, δ, and ι; full normalization; and definitional equality.

function Whnf(t)
  loop
    write t as a spine $h\ a_1 \cdots a_n$
    if $h = \lambda(x{:}A).\ b$ and $n \geq 1$ **then** ▷ β
      $t \leftarrow (b[x \mapsto a_1])\ a_2 \cdots a_n$
    else if $h = c$ and $(c : C := d) \in \Sigma$ **then** ▷ δ
      $t \leftarrow d\ a_1 \cdots a_n$
    else if $h = c$ is a recursor with major position $p \leq n$ **then** ▷ ι
      $k\ b_1 \cdots b_j \leftarrow$ Whnf(a_p)
      if k is a constructor with a rule r of c, and k has q parameters **then**
        $t \leftarrow r.rhs\ a_1 \cdots a_{p-1}\ b_{q+1} \cdots b_j\ a_{p+1} \cdots a_n$
      else return t ▷ stuck, e.g. on a variable
    else return t

function Normalize(t)
  $t \leftarrow$ Whnf(t)
  if $t = \Pi(x{:}A).\ B$ **then return** $\Pi(x{:}$Normalize(A)$).$ Normalize(B)
  else if $t = \lambda(x{:}A).\ b$ **then return** $\lambda(x{:}$Normalize(A)$).$ Normalize(b)
  else write t as a spine $h\ a_1 \cdots a_n$ and **return** h Normalize(a_1) $\cdots$ Normalize(a_n)

function IsDefEq(s, t)
  return $\cdot\,;\cdot \vdash$ Normalize(s) $=_\alpha$ Normalize(t) ▷ Fig. 5

Whnf reduces only at the head of a term, until no rule applies there; the result is the *weak head normal form*. It never looks inside a λ, a Π, or the arguments of a stuck head. This is enough when

a caller only needs the outermost shape of a type, such as a Π in the application case of Infer, and it is cheaper than full normalization. Normalize first computes the weak head normal form and then normalizes every immediate subterm. For example, with $t = \lambda(h{:}Id\ P).\ h$ and Id as above,

$$\textsc{Whnf}(t) = t, \qquad \textsc{Normalize}(t) = \lambda(h{:}P).\ h,$$

because the head of t is a λ with no arguments, while Normalize continues into the annotation $Id\ P$. Likewise, $\textsc{Whnf}(Id\ (Id\ P \to P)) = Id\ P \to P$, which already shows a Π although its domain is not yet normal.

The three cases of Whnf have the following properties.

- The ι case contains no reference to Nat, Eq, or Tree. It reads the major position and the rules from the signature (§9.5). It is written once; after that, each new inductive type adds rows of data, not code.
- The major premise is put in weak head normal form before the rules are consulted, because it may become a constructor only after computation. For $2 + 1$ in §7.2, the major premise is the numeral $1 = \text{succ zero}$, already a constructor; for *double* $(1 + 1)$, it is $1 + 1$, which Whnf turns into $\text{succ}\ (1 + 0)$ first.
- If the major premise does not become a constructor, for instance because it is a variable, the term is stuck and stays as it is; this is why $0 + n$ does not reduce (§7.6).

IsDefEq normalizes fully for simplicity, and our implementation does the same. Practical kernels compare lazily instead: they reduce both sides to weak head normal form, compare the heads, and recurse into the arguments only when needed, which avoids normalizing large terms that differ at the head. Termination of all three functions for well-typed terms is strong normalization.

10 UNIVERSES: WHY Sort u?

§4 showed that the typing rules implement a logic, and that the system is acceptable only if it is consistent. This section asks what the type of Sort 0 should be. It shows that the simplest choice of universes, a single universe that contains itself, makes the system inconsistent (§10.2), and that the hierarchy $\text{Sort } 0 : \text{Sort } 1 : \cdots$ with an impredicative bottom universe, as in Lean, avoids the problem (§10.3).

10.1 What Is the Type of Sort 0?

A proposition variable appears in the type theory as an assumption $P : \text{Sort } 0$. It can occur in two places: in a context, as in the left judgment below, or bound by λ or Π, as in the right one:

$$P : \text{Sort } 0 \vdash \lambda(h{:}P).\ h : P \to P \qquad \text{versus} \qquad \vdash \lambda(P{:}\text{Sort } 0).\ \lambda(h{:}P).\ h : \Pi(P{:}\text{Sort } 0).\ P \to P.$$

The left judgment proves $P \to P$ for one fixed P supplied from outside. The right one is a single closed term that proves it for every P, and it can be applied later to any particular proposition; this is what the ∀ rows of Fig. 2 need.

Both forms require Sort 0 to be a type. C-Ext, which adds $P : \text{Sort } 0$ to a context, and T-Lam and T-Pi, which bind $P : \text{Sort } 0$, all have the premise $\Gamma \vdash A : \text{Sort } u$ for the type A of the new variable. With $A = \text{Sort } 0$, this premise reads $\Gamma \vdash \text{Sort } 0 : \text{Sort } u$ for some u: the universe of propositions must itself have a type. §10.2 shows that the simplest answer, that Sort 0 is its own type, makes the system inconsistent. §10.3 then presents Lean's answer, a hierarchy of universes.

10.2 Type Is Not a Type

Suppose we replace all universes by a single universe Type that contains itself:

$$\frac{\vdash \Gamma \text{ ok}}{\Gamma \vdash \text{Type} : \text{Type}} \qquad \frac{\Gamma \vdash A : \text{Type} \qquad \Gamma, x : A \vdash B : \text{Type}}{\Gamma \vdash \Pi(x{:}A).\, B : \text{Type}}$$

Translate $\forall P.\, \varphi$ as $\Pi(P{:}\text{Type}).\, \varphi^*$. Then P ranges over a collection that contains Type itself and every type built from it. Such a collection is the type-theoretic analogue of the set of all sets, and naive set theory shows that such collections lead to contradictions. The best-known example is *Russell's paradox*: for $R = \{\, x \mid x \notin x \,\}$, both $R \in R$ and $R \notin R$ lead to a contradiction. Type theory has no membership test like $x \notin x$, so Russell's argument does not carry over word for word. Girard instead adapted a related paradox about the collection of all well-orderings, due to Burali-Forti [7], and Hurkens simplified the resulting term [8]. The result is a closed term t_{H} with

$$\vdash t_{\mathrm{H}} : \Pi(P{:}\text{Type}).\, P,$$

so the system is inconsistent: $\bot$ has no proof, yet $\bot^*$ has a term, and a kernel built on these rules accepts a proof of every statement. The essential ingredient is a type U that quantifies over all of Type while being a member of Type, which lets U contain a copy of its own double power set, something no set can do. Appendix C gives t_{H} in full and explains how it works.

The damage reaches the algorithm as well. Subject reduction still holds, so if t_{H} had a normal form, it would be a normal closed term of type $\Pi(p{:}\text{Type}).\, p$. The part of the proof of Theorem 7 (Appendix D) that analyses normal terms does not use strong normalization and carries over to Type : Type; it shows that no such normal term exists. Hence t_{H} has no normal form, strong normalization fails, and Normalize (Algorithm 2) runs forever on it.

10.3 A Hierarchy with an Impredicative Bottom

The second answer is that Sort 0 lives in a larger universe, which lives in a larger one still:

$$\frac{\vdash \Gamma \text{ ok}}{\Gamma \vdash \text{Sort } u : \text{Sort } (u+1)}\ (\text{T-Sort}) \qquad \text{so} \qquad \text{Sort } 0 : \text{Sort } 1 : \text{Sort } 2 : \cdots .$$

No universe contains itself. A Π type takes an argument from one universe and returns a type in another, so T-Pi must also say which universe the Π type itself belongs to. Lean's answer uses the function imax:

$$\frac{\Gamma \vdash A : \text{Sort } u \qquad \Gamma, x : A \vdash B : \text{Sort } v}{\Gamma \vdash \Pi(x{:}A).\, B : \text{Sort } \operatorname{imax}(u,v)}\ (\text{T-Pi}) \qquad \operatorname{imax}(u,v) = \begin{cases} 0 & \text{if } v = 0, \\ \max(u,v) & \text{otherwise.} \end{cases}$$

The case split in imax makes a Π type impredicative when its body is a proposition, and predicative otherwise.

Propositions are impredicative. If the body B is a proposition ($v = 0$), the whole Π is a proposition, whatever it quantifies over. In particular $(\forall P.\, \varphi)^* = \Pi(P{:}\text{Sort } 0).\, \varphi^*$ is in Sort 0, although it quantifies over all of Sort 0. A universe that contains types quantifying over itself is called *impredicative*. Because every translated formula lies in Sort 0, the premise $\psi^* : \text{Sort } 0$ of ∀-elim always holds, so a universal statement can be instantiated with any proposition, even with itself. For example, with $id = \lambda(P{:}\text{Sort } 0).\, \lambda(h{:}P).\, h$,

$$id : \Pi(P{:}\text{Sort } 0).\, P \to P \qquad \text{and} \qquad \Pi(P{:}\text{Sort } 0).\, P \to P : \text{Sort } 0,$$

so $id\ (\Pi(P{:}\text{Sort } 0).\, P \to P)\ id$ is well typed, with type $\Pi(P{:}\text{Sort } 0).\, P \to P$. The same case makes $\forall n : \text{Nat},\ n + 0 = n$ a proposition, even though Nat lives in Sort 1; §7.2 relies on this for induction.

Types are predicative. If the body is not a proposition ($v \geq 1$), the Π lives in the larger of the two universes. A type that quantifies over Sort u then lives in Sort $(\max(u+1, v))$, which is never Sort u:

$$\dfrac{\dfrac{}{\Gamma \vdash \mathrm{Sort}\ u : \mathrm{Sort}\ (u+1)}\ \textsc{T-Sort} \qquad \Gamma, x : \mathrm{Sort}\ u \vdash B : \mathrm{Sort}\ v}{\Gamma \vdash \Pi(x{:}\mathrm{Sort}\ u).\, B : \mathrm{Sort}\ (\max(u+1, v))}\ \textsc{T-Pi}$$

This property is called *predicativity*. For example, the identity on types $\lambda(A{:}\mathrm{Sort}\ 1).\, \lambda(a{:}A).\, a$ has type $\Pi(A{:}\mathrm{Sort}\ 1).\, A \to A$, which lives in Sort 2, so it cannot be applied to its own type; a copy at Sort 2 can. Each level holds the level beneath it, and in practice a few levels suffice; higher levels appear when one quantifies over large collections, such as the category of all small categories.

Why this is safe. Impredicativity looks like the self-reference that §10.2 warned against, but it is confined: Sort 0 contains types that quantify over Sort 0, yet Sort 0 is not a member of itself. Coquand showed that one impredicative universe at the bottom is consistent [2], whereas Girard's paradox can already be derived once two levels are impredicative [8]. With these rules the system is consistent and strongly normalizing (§11).

10.4 Comparison with Lean

In Lean [9], Sort 0 is Prop, Sort $(u+1)$ is Type u, and Π types are assigned universes with imax exactly as above. Two differences remain. First, Prop in Lean has *definitional proof irrelevance*: any two proofs of the same proposition are definitionally equal. Our conversion does not identify proofs; adding this would make conversion depend on types, so the simple "normalize and compare" check of §9.4 would no longer suffice. Second, Lean has *universe polymorphism*, so definitions can be stated once for all levels u, and imax then matters for level variables as well. In our system every level is a concrete number.

Table 1. Design choices for universes.

Design	Quantifies over	Drawback
No universes	values only	cannot say "for every proposition"
Type : Type	everything	inconsistent
Two sorts $* : \Box$	$*$ only	$\Box$ itself has no type
Sort u : Sort $(u+1)$ with max	any level	$\forall$ over propositions is not a proposition
Sort u : Sort $(u+1)$ with imax (Lean)	any level	normalization is harder to prove

11 PROPERTIES

We state, without proof, the standard properties of the core system of §§2–6, with the universes of §10 and without the constants of §7. It is a Pure Type System: the Calculus of Constructions [2] with a hierarchy of predicative universes above its impredicative Sort 0, close to Luo's Extended Calculus of Constructions [3]. The first three properties below hold for every functional Pure Type System [5]. Strong normalization is much harder in the presence of impredicativity and needs Girard's method of reducibility candidates [2, 3, 7]. The end of this section revisits them for inductive types.

Theorem 3 (Uniqueness of types). *If $\Gamma \vdash t : A$ and $\Gamma \vdash t : B$, then $A \equiv B$.*

Theorem 4 (Type correctness). *If $\Gamma \vdash t : A$, then either $A = \mathrm{Sort}\ u$ for some u, or $\Gamma \vdash A : \mathrm{Sort}\ u$ for some u.*

Theorem 5 (Subject reduction). *If* $\Gamma \vdash t : A$ *and* $t \to_\beta t'$, *then* $\Gamma \vdash t' : A$.

Theorem 6 (Strong normalization). *If* $\Gamma \vdash t : A$, *then every sequence of* β*-reductions starting from* t *is finite.*

Together these give the two guarantees we want from a kernel. *Decidability*: Algorithm 1 terminates on every input, so whether a proof is correct can always be determined. *Consistency*: no closed term proves False.

Theorem 7 (Consistency). *There is no closed term* t *with* $\cdot \vdash t : \Pi(P{:}\mathrm{Sort}\ 0).\ P$.

By strong normalization and subject reduction, such a term would have a normal form of the same type. A closed normal term whose type is a Π must be a λ: it cannot be a variable or start with one, since no variable is in scope, and universes and Π types have a universe as their type. So the normal form is $\lambda(P{:}\mathrm{Sort}\ 0).\ s$ with $P : \mathrm{Sort}\ 0 \vdash s : P$ and s normal. A normal term is a universe, a Π, a λ, or a variable applied to arguments, and none of these can have type P in that context: the only variable available is P itself, whose type is $\mathrm{Sort}\ 0$, not P. Appendix D gives the full proof, including the facts about typing and conversion that this argument uses.

With inductive types. The ι-rules of §7 do not overlap with each other or with β, so reduction remains confluent. Subject reduction, strong normalization, and consistency are known for the Calculus of Inductive Constructions [4], of which our system is a small fragment. The short consistency argument above no longer suffices by itself, because normal forms now include stuck recursor applications; consistency follows from strong normalization by a more careful analysis of normal forms. Lean's full theory, with definitional proof irrelevance and general inductive types, has subtler metatheory [1].

REFERENCES

[1] M. Carneiro. *The Type Theory of Lean.* Master's thesis, Carnegie Mellon University, 2019.
[2] T. Coquand and G. Huet. The calculus of constructions. *Information and Computation*, 76(2–3), 1988.
[3] Z. Luo. *An Extended Calculus of Constructions.* PhD thesis, University of Edinburgh, 1990.
[4] B. Werner. *Une théorie des constructions inductives.* PhD thesis, Université Paris 7, 1994.
[5] H. P. Barendregt. Lambda calculi with types. In *Handbook of Logic in Computer Science*, 1992.
[6] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation. *Indagationes Mathematicae*, 1972.
[7] J.-Y. Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur.* PhD thesis, 1972.
[8] A. J. C. Hurkens. A simplification of Girard's paradox. *TLCA*, 1995.
[9] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *CADE*, 2021.

A PROOF OF THEOREM 1

The precise statement. Theorem 1 leaves implicit which proposition variables are in scope. For the proof we make them explicit and write $\Theta; \Delta \vdash \varphi$, where Θ is a list of distinct proposition variables and every proposition variable free in Δ or φ belongs to Θ. The rules of Fig. 2 then read

$$\frac{\varphi \in \Delta}{\Theta; \Delta \vdash \varphi} \qquad \frac{\Theta; \Delta, \varphi \vdash \psi}{\Theta; \Delta \vdash \varphi \to \psi} \qquad \frac{\Theta; \Delta \vdash \varphi \to \psi \qquad \Theta; \Delta \vdash \varphi}{\Theta; \Delta \vdash \psi} \qquad \frac{\Theta, P; \Delta \vdash \varphi \qquad P \notin \Theta}{\Theta; \Delta \vdash \forall P.\ \varphi}$$

$$\frac{\Theta; \Delta \vdash \forall P.\ \varphi \qquad \mathrm{FV}(\psi) \subseteq \Theta}{\Theta; \Delta \vdash \varphi[P \mapsto \psi]}$$

The side condition $P \notin \mathrm{FV}(\Delta)$ of Fig. 2 is now implied by $P \notin \Theta$. Formulas are taken up to α-equivalence, and $\varphi^* \to \psi^*$ stands for $\Pi(x{:}\varphi^*).\ \psi^*$ with x fresh.

The context Δ^* of §4 lists the proposition variables first and the hypotheses after them. The proof has to extend contexts on the right, so it is convenient to allow any order.

Definition 5 (Representation). *A context* Γ represents $(\Theta;\Delta)$ *if* $\vdash \Gamma$ ok *and* Γ *consists of one assumption* $P : \mathrm{Sort}\ 0$ *for each* $P \in \Theta$ *and one assumption* $h : \chi^*$ *for each* $\chi \in \Delta$*, in some order.*

Theorem 8 (Theorem 1, precise form). *If* $\Theta;\Delta \vdash \varphi$ *and* Γ *represents* $(\Theta;\Delta)$*, then* $\Gamma \vdash t : \varphi^*$ *for some term* t*.*

The context Δ^* represents $(\Theta;\Delta)$ when Θ lists the free proposition variables; it is well formed by Lemma 9 below and C-Ext. So Theorem 1 is the special case $\Gamma = \Delta^*$.

Facts about the translation. Translation commutes with substitution: $(\varphi[P \mapsto \psi])^* = \varphi^*[P \mapsto \psi^*]$. This is a routine induction on φ.

Lemma 9 (Formulas are propositions). *If* $\vdash \Gamma$ ok *and* $(P : \mathrm{Sort}\ 0) \in \Gamma$ *for every* $P \in \mathrm{FV}(\varphi)$*, then* $\Gamma \vdash \varphi^* : \mathrm{Sort}\ 0$.

Proof. By induction on φ. For P, use T-Var. For $\varphi \to \psi$, the induction hypothesis gives $\Gamma \vdash \varphi^* : \mathrm{Sort}\ 0$, so $\Gamma, x : \varphi^*$ is well formed by C-Ext for a fresh x. The induction hypothesis for ψ in that context gives Sort 0, and T-Pi gives Sort $\mathrm{imax}(0,0) = \mathrm{Sort}\ 0$. For $\forall Q.\ \chi$, rename Q so that $Q \notin \mathrm{dom}(\Gamma)$. T-Sort gives $\Gamma \vdash \mathrm{Sort}\ 0 : \mathrm{Sort}\ 1$, so $\Gamma, Q : \mathrm{Sort}\ 0$ is well formed, and T-Pi with the induction hypothesis for χ gives Sort $\mathrm{imax}(1,0) = \mathrm{Sort}\ 0$. □

The last case is where impredicativity is used. With max in place of imax, $(\forall Q.\ \chi)^*$ would live in Sort 1, and the $\forall$-elim case below would hold only for formulas ψ without $\forall$.

Proof of Theorem 8. By induction on the derivation of $\Theta;\Delta \vdash \varphi$, for all Γ at once. Each case builds the typing derivation shown in the right column of Fig. 2.

- *Hypothesis.* Γ contains $h : \varphi^*$, so $\Gamma \vdash h : \varphi^*$ by T-Var.
- $\to$*-intro.* By Lemma 9, $\Gamma \vdash \varphi^* : \mathrm{Sort}\ 0$. For a fresh h, the context $\Gamma, h : \varphi^*$ is well formed and represents $(\Theta;\Delta,\varphi)$. The induction hypothesis gives $\Gamma, h : \varphi^* \vdash t : \psi^*$, and T-Lam gives $\Gamma \vdash \lambda(h{:}\varphi^*).\ t : \Pi(h{:}\varphi^*).\ \psi^*$. This type is $\varphi^* \to \psi^*$, since the free variables of ψ^* are proposition variables and h is not one of them.
- $\to$*-elim.* The induction hypotheses give $\Gamma \vdash f : \Pi(x{:}\varphi^*).\ \psi^*$ with $x \notin \mathrm{FV}(\psi^*)$ and $\Gamma \vdash a : \varphi^*$. T-App gives $\Gamma \vdash f\ a : \psi^*[x \mapsto a] = \psi^*$.
- $\forall$*-intro.* Renaming P if necessary, assume $P \notin \mathrm{dom}(\Gamma)$; a routine induction shows that renaming a variable of Θ to a fresh one preserves derivations. By T-Sort and C-Ext, $\Gamma, P : \mathrm{Sort}\ 0$ is well formed, and it represents $(\Theta, P;\Delta)$. The induction hypothesis gives $\Gamma, P : \mathrm{Sort}\ 0 \vdash t : \varphi^*$, and T-Lam gives $\Gamma \vdash \lambda(P{:}\mathrm{Sort}\ 0).\ t : \Pi(P{:}\mathrm{Sort}\ 0).\ \varphi^*$.
- $\forall$*-elim.* The induction hypothesis gives $\Gamma \vdash f : \Pi(P{:}\mathrm{Sort}\ 0).\ \varphi^*$, and Lemma 9 gives $\Gamma \vdash \psi^* : \mathrm{Sort}\ 0$. T-App gives $\Gamma \vdash f\ \psi^* : \varphi^*[P \mapsto \psi^*] = (\varphi[P \mapsto \psi])^*$. □

B PROOF OF THEOREM 2

Both sides of Theorem 2 turn out to say the same thing: s and t have the same de Bruijn representation. We first define that representation, and then relate it to the algorithm (Lemma 10) and to the definition (Lemma 13).

Nameless terms. A *nameless term* is built like a term, except that binders carry no names and a bound variable is written as a number $\#i$:

$$d ::= \mathrm{Sort}\ u \mid \#i \mid x \mid \Pi\, d.\, d \mid \lambda\, d.\, d \mid d\ d$$

Free variables keep their names. For a list of names E, the nameless form $\lfloor t \rfloor_E$ of a term t that occurs under the binders E is

$$\lfloor \text{Sort } u \rfloor_E = \text{Sort } u \qquad \lfloor x \rfloor_E = \begin{cases} \#idx(E, x) & \text{if } x \in E, \\ x & \text{otherwise,} \end{cases}$$
$$\lfloor \Pi(x{:}A).\, B \rfloor_E = \Pi\, \lfloor A \rfloor_E .\, \lfloor B \rfloor_{E,x} \qquad \lfloor \lambda(x{:}A).\, t \rfloor_E = \lambda\, \lfloor A \rfloor_E .\, \lfloor t \rfloor_{E,x}$$
$$\lfloor t_1\ t_2 \rfloor_E = \lfloor t_1 \rfloor_E\ \lfloor t_2 \rfloor_E .$$

Equality of nameless terms is plain syntactic equality. Note that $\lfloor t \rfloor_E$ has the same outermost shape as t, except that a variable may become either a number or a name. Constants (§7.1) are left unchanged, like universes, and are compared by name; they add only trivial cases below.

Lemma 10 (The algorithm compares nameless forms). *For all E_1, E_2, s, and t: $E_1; E_2 \vdash s =_\alpha t$ is derivable if and only if $\lfloor s \rfloor_{E_1} = \lfloor t \rfloor_{E_2}$.*

Proof. By induction on s, for all t, E_1, and E_2. For each shape of s, exactly one rule of Fig. 5 can conclude the judgment, and its premises match the equations for $\lfloor \cdot \rfloor$.

- If $s = \text{Sort } u$, both sides hold exactly when $t = \text{Sort } u$, by α-Sort.
- If $s = x$, the nameless forms can be equal only if t is a variable y. If $x \in E_1$, then $\lfloor x \rfloor_{E_1} = \#idx(E_1, x)$, and equality holds exactly when $y \in E_2$ and $idx(E_2, y) = idx(E_1, x)$, which is the premise of α-Bound. If $x \notin E_1$, then $\lfloor x \rfloor_{E_1} = x$, and equality holds exactly when $y \notin E_2$ and $y = x$, which is the premise of α-Free. A number never equals a name, matching the absence of a rule for a variable that is bound on one side only.
- If $s = \Pi(x{:}A).\, B$, equality holds exactly when $t = \Pi(y{:}A').\, B'$ with $\lfloor A \rfloor_{E_1} = \lfloor A' \rfloor_{E_2}$ and $\lfloor B \rfloor_{E_1,x} = \lfloor B' \rfloor_{E_2,y}$. By the induction hypothesis, these are the premises of α-Pi. The cases λ and application are the same. □

The definition of $=_\alpha$ involves substitution, so we need two facts about how nameless forms behave under renaming. The first says that $\lfloor t \rfloor_E$ depends on E only through the free variables of t.

Lemma 11 (Agreement). *If $idx(E_1, v) = idx(E_2, v)$ for every $v \in \text{FV}(t)$, with both sides possibly undefined, then $\lfloor t \rfloor_{E_1} = \lfloor t \rfloor_{E_2}$.*

Proof. By induction on t, for all E_1 and E_2. The variable case is the assumption. Under a binder x, the lists E_1, x and E_2, x again agree on the free variables of the body: x has position 0 in both, and any other free variable has its position increased by one in both, or stays undefined in both. □

The second says that renaming a binder does not change the nameless form. We use that substituting a variable for a variable preserves the size of a term, even when binders are renamed; this is a routine induction.

Lemma 12 (Renaming). *Let $y \neq x$, $y \notin \text{FV}(B)$, and $x, y \notin E'$. Then $\lfloor B \rfloor_{E,x,E'} = \lfloor B[x \mapsto y] \rfloor_{E,y,E'}$ for every list E.*

Proof. By induction on the size of B, for all E and E'. Write $L_x = E, x, E'$ and $L_y = E, y, E'$; they differ only in one entry, at the same position.

- If $B = x$, then $B[x \mapsto y] = y$. Since $x, y \notin E'$, both $idx(L_x, x)$ and $idx(L_y, y)$ equal the length of E'.
- If $B = v \neq x$, then $v \neq y$ because $y \notin \text{FV}(B)$, and $B[x \mapsto y] = v$. The position of v is the same in L_x and L_y, or undefined in both.
- Universes and applications are immediate from the definitions and the induction hypothesis.

- Let $B = \Pi(w{:}C).\, D$; the case of λ is the same. The annotation C is handled by the induction hypothesis in each case below, so we consider only the body. There are three cases, following the definition of substitution.
  - $w = x$. Then $B[x \mapsto y] = \Pi(x{:}C[x \mapsto y]).\, D$, and we need $\lfloor D \rfloor_{L_x,x} = \lfloor D \rfloor_{L_y,x}$. We apply Lemma 11. The variable x has position 0 in both lists. Any other $v \in \mathrm{FV}(D)$ differs from y, since $y \notin \mathrm{FV}(B)$, so its position is the same in both lists, or undefined in both.
  - $w \notin \{x, y\}$. Then $B[x \mapsto y] = \Pi(w{:}C[x \mapsto y]).\, D[x \mapsto y]$. The induction hypothesis for D, with E' extended by w, gives $\lfloor D \rfloor_{L_x,w} = \lfloor D[x \mapsto y] \rfloor_{L_y,w}$.
  - $w = y$. The binder would capture y, so substitution renames it to a fresh $z \notin \mathrm{FV}(D) \cup \{x, y\}$: $B[x \mapsto y] = \Pi(z{:}C[x \mapsto y]).\, (D[y \mapsto z])[x \mapsto y]$. The induction hypothesis for D, renaming its last binder y to z, gives $\lfloor D \rfloor_{L_x,y} = \lfloor D[y \mapsto z] \rfloor_{L_x,z}$. The induction hypothesis for $D[y \mapsto z]$, which has the size of D and no free y, with E' extended by z, gives $\lfloor D[y \mapsto z] \rfloor_{L_x,z} = \lfloor (D[y \mapsto z])[x \mapsto y] \rfloor_{L_y,z}$. □

Lemma 13 (The definition compares nameless forms). *If $s =_\alpha t$ in the sense of Definition 3, then $\lfloor s \rfloor_E = \lfloor t \rfloor_E$ for every E. Conversely, if $\lfloor s \rfloor_E = \lfloor t \rfloor_E$ for some E, then $s =_\alpha t$.*

Proof. For the first part, let $s \sim t$ mean that $\lfloor s \rfloor_E = \lfloor t \rfloor_E$ for every E. This relation is an equivalence, and it is a congruence, because $\lfloor \cdot \rfloor$ is defined constructor by constructor. It contains the defining equation of Definition 3: if $y \notin \mathrm{FV}(B)$, then $\lfloor \Pi(x{:}A).\, B \rfloor_E = \Pi\, \lfloor A \rfloor_E.\, \lfloor B \rfloor_{E,x}$ and $\lfloor \Pi(y{:}A).\, B[x \mapsto y] \rfloor_E = \Pi\, \lfloor A \rfloor_E.\, \lfloor B[x \mapsto y] \rfloor_{E,y}$, and the bodies are equal by Lemma 12 with E' empty (the case $y = x$ is trivial). Since $=_\alpha$ is the smallest congruence containing that equation, $s =_\alpha t$ implies $s \sim t$.

For the converse, we use induction on the size of s, for all t and E.

- If s is a variable, so is t. If both nameless forms are $\#i$, both variables are the entry at position i of E, so $s = t$. If both are names, then $s = t$ directly.
- Universes and applications follow from the induction hypothesis and the congruence of $=_\alpha$.
- Let $s = \Pi(x{:}A).\, B$, so that $t = \Pi(y{:}A').\, B'$ with $\lfloor A \rfloor_E = \lfloor A' \rfloor_E$ and $\lfloor B \rfloor_{E,x} = \lfloor B' \rfloor_{E,y}$. The induction hypothesis gives $A =_\alpha A'$. For the bodies, choose $z \notin \mathrm{FV}(B) \cup \mathrm{FV}(B') \cup \{x, y\}$. Lemma 12 with E' empty gives $\lfloor B \rfloor_{E,x} = \lfloor B[x \mapsto z] \rfloor_{E,z}$ and $\lfloor B' \rfloor_{E,y} = \lfloor B'[y \mapsto z] \rfloor_{E,z}$, so $\lfloor B[x \mapsto z] \rfloor_{E,z} = \lfloor B'[y \mapsto z] \rfloor_{E,z}$. Since $B[x \mapsto z]$ has the size of B, the induction hypothesis gives $B[x \mapsto z] =_\alpha B'[y \mapsto z]$. Then

  $$\Pi(x{:}A).\, B \;=_\alpha\; \Pi(z{:}A).\, B[x \mapsto z] \;=_\alpha\; \Pi(z{:}A').\, B'[y \mapsto z] \;=_\alpha\; \Pi(y{:}A').\, B',$$

  by the defining equation, by congruence, and by the defining equation read backwards. The case of λ is the same. □

Proof of Theorem 2. If $s =_\alpha t$, then $\lfloor s \rfloor_\cdot = \lfloor t \rfloor_\cdot$ by Lemma 13, and $\cdot\,;\cdot \vdash s =_\alpha t$ by Lemma 10. Conversely, if $\cdot\,;\cdot \vdash s =_\alpha t$, then $\lfloor s \rfloor_\cdot = \lfloor t \rfloor_\cdot$ by Lemma 10, and $s =_\alpha t$ by Lemma 13. □

C HURKENS' PARADOX

This appendix gives the term t_{H} of §10.2 in full (Fig. 6) and explains how it works. We checked it with the kernel of `chapters/01`, with T-Sort changed to Type : Type: Infer returns exactly $\Pi(p{:}\mathrm{Type}).\, p$ for t_{H}, while the unchanged kernel rejects the term.

How the term works. Read $\wp S = S \to \mathrm{Type}$ as the type of predicates on S, that is, of "subsets" of S. The type U is built to contain a copy of its own double power set $\wp\wp U$: $\tau : \wp\wp U \to U$ packs a family of subsets of U into one element of U, and $\sigma : U \to \wp\wp U$ unpacks it, with

$$\sigma\ (\tau\ t)\ p \;\equiv\; t\ (\lambda(x{:}U).\, p\ (\tau\ (\sigma\ x)))$$

$$\bot = \Pi(p{:}\mathrm{Type}).\, p, \qquad \neg\varphi = \varphi \to \bot, \qquad \wp S = S \to \mathrm{Type}$$
$$U = \Pi(X{:}\mathrm{Type}).\, ((\wp\wp X \to X) \to \wp\wp X)$$
$$\tau = \lambda(t{:}\wp\wp U).\, \lambda(X{:}\mathrm{Type}).\, \lambda(f{:}\wp\wp X \to X).\, \lambda(p{:}\wp X).\, t\ (\lambda(x{:}U).\, p\ (f\ (x\ X\ f)))$$
$$\sigma = \lambda(s{:}U).\, s\ U\ \tau$$
$$\Delta = \lambda(y{:}U).\, \neg\,(\Pi(p{:}\wp U).\, \sigma\ y\ p \to p\ (\tau\ (\sigma\ y)))$$
$$\Omega = \tau\ (\lambda(p{:}\wp U).\, \Pi(x{:}U).\, \sigma\ x\ p \to p\ x)$$

$$t_{\mathrm{H}} = (\lambda(h_0 : \Pi(p{:}\wp U).\, (\Pi(x{:}U).\, \sigma\ x\ p \to p\ x) \to p\ \Omega).\\ h_0\ \Delta\ (\lambda(x{:}U).\, \lambda(h_2{:}\sigma\ x\ \Delta).\, \lambda(h_3{:}\Pi(p{:}\wp U).\, \sigma\ x\ p \to p\ (\tau\ (\sigma\ x))).\, h_3\ \Delta\ h_2\ (\lambda(p{:}\wp U).\, h_3\ (\lambda(y{:}U).\, p\ (\tau\ (\sigma\ y)))))\\ (\lambda(p{:}\wp U).\, h_0\ (\lambda(y{:}U).\, p\ (\tau\ (\sigma\ y)))))\\ (\lambda(p{:}\wp U).\, \lambda(h_1{:}\Pi(x{:}U).\, \sigma\ x\ p \to p\ x).\, h_1\ \Omega\ (\lambda(x{:}U).\, h_1\ (\tau\ (\sigma\ x))))$$

Fig. 6. Hurkens' paradox in a system with Type : Type. The first six lines are abbreviations, expanded in place; t_{H} has type $\bot$.

by β-reduction alone. In set theory no set contains a copy of its power set (Cantor's theorem), let alone of its double power set. U can, because it quantifies over all of Type while being a member of Type.

With σ we can imitate an ordering on U. Read $\sigma\ x\ p$ as "p holds for every element below x", call p *inductive* if $\Pi(x{:}U).\, \sigma\ x\ p \to p\ x$, and call x *well-founded* if every inductive p holds of x. The element Ω plays the role of "the collection of all well-founded elements", like the set of all ordinals in Burali-Forti's paradox. The three arguments in t_{H} then say the following.

- The last line proves that Ω is well-founded. Its type is the type of h_0.
- $h_0\ \Delta\ (\cdots)$ proves that the predicate Δ, "y is not below itself", is inductive, and applies the well-foundedness of Ω to it: Ω is not below itself. (Strictly, $\Delta\ y$ speaks of $\tau\ (\sigma\ y)$, which behaves like y.)
- The argument $\lambda(p{:}\wp U).\, h_0\ (\cdots)$ proves, using the equation above, that Ω is below itself.

Applying the second to the third yields a term of type $\bot$. Type : Type is used in one place: σ applies $s : U$ to U itself, which requires U : Type although U quantifies over Type. In the hierarchy of §10.3, U lives in a higher universe than the one it quantifies over, so this application is ill typed.

D PROOF OF THEOREM 7

The proof concerns the core system of §11: the rules of Fig. 1 with the universes of §10, without the constants of §7. It combines strong normalization with an analysis of the shapes of normal terms.

Facts about the type system. Besides subject reduction and strong normalization (§11), the proof uses the following standard properties of Pure Type Systems [5]. Subject reduction extends from one step to any number of steps by induction on the number of steps.

Lemma 14.

(a) Generation. *If* $\Gamma \vdash t : T$, *then, depending on the shape of* t*:*
- *if* $t = x$, *then* $(x : A) \in \Gamma$ *and* $T \equiv A$ *for some* A*;*
- *if* $t = \mathrm{Sort}\ u$, *then* $T \equiv \mathrm{Sort}\ (u + 1)$*;*
- *if* $t = \Pi(x{:}A).\, B$, *then* $T \equiv \mathrm{Sort}\ w$ *for some* w*;*
- *if* $t = \lambda(x{:}A).\, s$, *then* $\Gamma, x : A \vdash s : B$ *and* $T \equiv \Pi(x{:}A).\, B$ *for some* B*;*
- *if* $t = f\ a$, *then* $\Gamma \vdash f : \Pi(x{:}A).\, B$ *and* $\Gamma \vdash a : A$ *for some* x, A, *and* B.

(b) Confluence. *If $s \equiv t$, then s and t reduce to a common term. Consequently:* $\Pi(x{:}A).\, B \equiv \Pi(y{:}A').\, B'$ *implies* $A \equiv A'$ *and* $B[x \mapsto z] \equiv B'[y \mapsto z]$ *for a fresh z; and no two of a variable, a universe, and a Π are convertible.*

(c) Substitution. *If $s \equiv t$, then $s[z \mapsto a] \equiv t[z \mapsto a]$.*

The consequences in (b) hold because a variable and a universe do not reduce at all, a Π reduces only to a Π, and two terms with a common reduct must therefore have the same outermost shape.

Shapes of normal terms. Recall from §9.6 that every term is a spine $h\ a_1 \cdots a_n$ whose head h is not an application.

Lemma 15 (Normal forms). *Let n be a normal term with $\Gamma \vdash n : T$. Then n is a universe, a Π, a λ, or a* neutral *term $x\ a_1 \cdots a_k$ with $k \geq 0$ whose head x is a variable in* $\mathrm{dom}(\Gamma)$. *Moreover, if $k \geq 1$, then $\Gamma \vdash x : C$ for some C that is convertible to a Π.*

Proof. Write $n = h\ a_1 \cdots a_k$. If $k = 0$, then $n = h$ is a universe, a Π, a λ, or a variable, and a variable is in $\mathrm{dom}(\Gamma)$ by generation.

Let $k \geq 1$. Applying the generation lemma for application k times, from the outside in, shows that every prefix $h\ a_1 \cdots a_i$ is typable, and in particular $\Gamma \vdash h : \Pi(y{:}A).\, B$ for some y, A, and B, from the innermost application $h\ a_1$. The head h is not a λ, since $h\ a_1$ would then be a β-redex inside the normal term n. It is not a universe or a Π either: by generation, its type would be convertible to a universe, and by confluence a universe is not convertible to $\Pi(y{:}A).\, B$. So h is a variable x, which is in $\mathrm{dom}(\Gamma)$ by generation, and $\Gamma \vdash x : \Pi(y{:}A).\, B$. □

Proof of Theorem 7. Suppose that $\cdot \vdash t : \Pi(P{:}\mathrm{Sort}\ 0).\, P$. By strong normalization, t reduces to a normal term n, and by subject reduction, $\cdot \vdash n : \Pi(P{:}\mathrm{Sort}\ 0).\, P$. We show that no such n exists, using Lemma 15 twice.

The normal form is a λ. By Lemma 15, n is a universe, a Π, a λ, or a neutral term.

- It is not neutral, because its head would be a variable in the empty context.
- It is not a universe or a Π, because generation would make $\Pi(P{:}\mathrm{Sort}\ 0).\, P$ convertible to a universe, which confluence rules out.

So $n = \lambda(x{:}A).\, s$. By generation, $x : A \vdash s : B$ and $\Pi(P{:}\mathrm{Sort}\ 0).\, P \equiv \Pi(x{:}A).\, B$ for some B. By confluence, $A \equiv \mathrm{Sort}\ 0$ and $P[P \mapsto z] \equiv B[x \mapsto z]$ for a fresh z, that is, $z \equiv B[x \mapsto z]$. Substituting x for z on both sides, which preserves convertibility by (c), gives $x \equiv B$. The body s is normal, because it is a subterm of n. In short, $x : A \vdash s : B$ with $A \equiv \mathrm{Sort}\ 0$ and $B \equiv x$.

The body cannot exist. Let $\Gamma = x : A$. By Lemma 15, s is a universe, a Π, a λ, or a neutral term.

- If s is a universe or a Π, generation makes B convertible to a universe; if s is a λ, it makes B convertible to a Π. But $B \equiv x$, and by confluence a variable is convertible to neither.
- If s is neutral, its head is a variable in $\mathrm{dom}(\Gamma) = \{x\}$, so $s = x\ a_1 \cdots a_k$. If $k = 0$, then $s = x$, and generation gives $B \equiv A$. Then $x \equiv B \equiv A \equiv \mathrm{Sort}\ 0$, but a variable is not convertible to a universe. If $k \geq 1$, Lemma 15 gives $\Gamma \vdash x : C$ with C convertible to a Π, and generation gives $C \equiv A \equiv \mathrm{Sort}\ 0$. Then a Π is convertible to a universe, which confluence rules out.

Every case leads to a contradiction, so no closed term has type $\Pi(P{:}\mathrm{Sort}\ 0).\, P$. □

In words: the only variable available is x, whose type is Sort 0. It can neither be returned, since Sort 0 is not x, nor be applied to anything, since Sort 0 is not a function type. Every other normal term has a universe or a Π as its type. The argument relies on strong normalization to reduce the question to normal terms; that step is where all the difficulty lies.